DMS 2.2

Debris disks around Main-sequence Stars

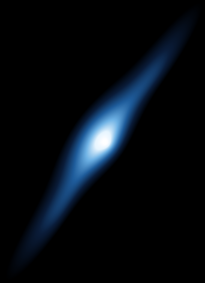
Reference manual and user guide



Scattered light, 1.6 µm
narrow ring, $i = 76°$



Polarised light Q_ϕ, 1.6 µm
same ring



Edge-on disk, 1.6 µm
$i = 87°$



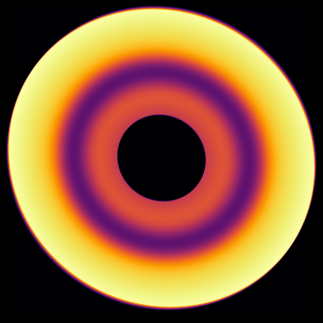
Two belts, 24 / 70 / 870 µm
warm inner, cold outer belt



Pericentre glow, 24 µm
$e = 0.3$, 1.5 arcsec beam



Apocentre glow, 870 µm
same ring and beam



Belt with a gap, 870 µm
user-defined density



Ring with a clump, 870 µm
user-defined density

Minjae G. Kim

Version 2.2.1 · November 2026

All images on this cover were computed with DMS 2.0.

DMS 2.2

Debris disks around Main-sequence Stars

Reference manual and user guide

Minjae G. Kim

Mullard Space Science Laboratory, University College London

Version 2.2.1 • November 2026

DMS computes spectral energy distributions, thermal emission maps, scattered-light images and linear-polarisation images of optically thin debris disks. The code is written in Fortran (free-form `.f90` source) with OpenMP and can be called from the command line or from Python. Part I of this manual describes the physical model, the equations and the numerical methods, including the adaptive refinement of volume elements used for the images. Part II explains how to install and run the code, lists every input parameter and output file, and documents the Python interface.

If you use DMS, please cite Kim et al. (2018, A&A 618, A38).

The images on the cover are described on page 48.

Contents

I Physics and numerical methods **4**

1 Introduction **4**
1.1 What DMS does . . . 4
1.2 Assumptions . . . 4
1.3 What is new in version 2.2 . . . 5
1.4 What is new in version 2.1 . . . 5
1.5 What is new in version 2.0 . . . 5
1.6 Notation . . . 5

2 Disk geometry **6**
2.1 Coordinate frames . . . 6
2.2 Radial density laws . . . 6
2.3 Eccentric belts . . . 7
2.4 Vertical structure . . . 8
2.5 Several belts . . . 8

3 The star **8**

4 Dust grains **8**
4.1 Size distribution . . . 8
4.2 Composite grains: effective-medium theory . . . 9
4.3 Distribution of hollow spheres . . . 10

5 Optical properties **10**
5.1 Mie theory and wavelength grids . . . 10
5.2 Scattering matrix . . . 11
5.3 Henyey–Greenstein phase functions . . . 11
5.4 Radiation pressure and blow-out . . . 11

6 Temperatures and sublimation **12**
6.1 Radiative equilibrium . . . 12
6.2 Temperature-dependent optical constants . . . 12
6.3 Sublimation . . . 13
6.4 Ice loss by photosputtering . . . 13

7 Emission, scattering and polarisation **14**
7.1 Contribution of one volume element . . . 14
7.2 Stokes parameters . . . 14
7.3 Global quantities . . . 15

8 Images and adaptive volume refinement **15**
8.1 Image grid . . . 15

8.2 Why refinement is needed . . . 15
8.3 Splitting criteria . . . 15
8.4 Implementation . . . 16

9 SEDs **17**

10 Simulated observations **17**

11 Verification **18**

12 Limitations **18**

II User guide **19**

13 Installation **19**
13.1 Prerequisites . . . 19
13.2 Getting the code and building . . . 20
13.3 Getting started with Jupyter . . . 21
13.4 Testing the installation . . . 21

14 Repository layout **22**

15 Running dms **23**
15.1 Command line . . . 23
15.2 What happens during a run . . . 23
15.3 A typical workflow . . . 23

16 Examples: a cookbook **25**
16.1 A first model in Python . . . 26
16.2 A ring from an input file: images and SED . . . 26
16.3 An eccentric ring: pericentre and apocentre glow . . . 27
16.4 Two belts, sublimation and simulated observations . . . 28
16.5 Polarimetric imaging with a PSF and a coronagraph . . . 30
16.6 Your own density distribution . . . 31
16.7 Composite and porous grains . . . 32
16.8 Non-spherical grains: hollow spheres . . . 32
16.9 Temperature-dependent optical constants . . . 33
16.10 Sublimation of ice . . . 34
16.11 Ice loss by photosputtering . . . 35
16.12 A grid of models in parallel . . . 36
16.13 Fitting an observed SED . . . 37

17 Input reference **37**
17.1 `&run`: what to compute and where to write it . . . 37
17.2 `&star` . . . 38

17.3 `&disk` . . . 38
17.4 `&image` . . . 39
17.5 `&wavelengths` . . . 39
17.6 `&dust` . . . 39
17.7 `&scattering` . . . 41
17.8 Optical data files . . . 41

18 Output files **41**
18.1 Images . . . 41
18.2 SED and tables . . . 42

19 Python interface **42**
19.1 Running a model . . . 43
19.2 The `Result` object . . . 43
19.3 Fitting and parallel runs . . . 44
19.4 Other modules . . . 44

20 Grids of models, composite grains and stellar atmospheres **44**
20.1 Batch runs: `dms.batch` . . . 44
20.2 Composite grains: `dms.emc` . . . 45
20.3 Model-atmosphere grids: `dms.stellar` . . . 45

21 Performance and accuracy **46**

22 Troubleshooting **46**

23 Extending dms **47**
23.1 Your own density distribution . . . 47
23.2 Adding an input parameter . . . 47

24 Citing, licence and acknowledgements **48**

About the cover **48**

References **49**

Part I
Physics and numerical methods

1 Introduction

1.1 What dms does

Debris disks consist of planetesimal belts and the dust that collisions between the planetesimals produce. The dust is optically thin at all wavelengths of interest: it absorbs stellar light, re-emits it in the infrared and submillimetre, and scatters it in the optical and near-infrared. DMS predicts what an observer sees from such a disk:

- the spectral energy distribution (SED) of thermal emission, scattered light and the star;
- images of thermal emission, scattered light, the Stokes parameters Q and U, the polarised intensity, the azimuthal Stokes parameters Q_ϕ and U_ϕ and the degree of polarisation, at any inclination and position angle;
- simulated observations, convolved with a point-spread function (PSF) and masked by a coronagraph;
- the fractional luminosity $f_\mathrm{d} = L_\mathrm{d}/L_\star$, the geometrical optical depth, the grain temperatures and the number of grains of every size.

The code was introduced by Kim et al. (2018), who used it for eccentric disks, and was applied to the detectability of water ice by Kim et al. (2019). Parts of it go back to the `debris/modim` code (Ertel et al. 2011, 2012). Optical properties of spheres are computed with `miex` (Wolf & Voshchinnikov 2004).

Related codes. Monte Carlo radiative-transfer codes such as MCFOST (Pinte et al. 2006), RADMC-3D (Dullemond et al. 2012) and Hyperion (Robitaille 2011) treat optically thick disks and multiple scattering, which DMS does not. For optically thin debris disks the transfer problem reduces to integrals along the line of sight. DMS evaluates these integrals directly, so its images and polarisation maps contain no photon noise, also for faint, highly inclined disks. This makes the code suited to large model grids and to fitting.

1.2 Assumptions

DMS makes the following assumptions. They hold for most debris disks; where they fail, the results should be checked with a full radiative-transfer code.

1. **Optically thin dust.** Every grain sees the unattenuated stellar radiation, and the radiation from the disk reaches the observer unattenuated. DMS prints the largest radial optical depth in the mid-plane and warns if it exceeds 0.1.
2. **Single scattering.** Light is scattered at most once; there is no heating of grains by the emission of other grains.
3. **Equilibrium temperatures.** Each grain is isothermal and in radiative equilibrium with the star, which is treated as a point source ($r \gg R_\star$). Stochastic heating of very small grains is not included.
4. **Prescribed dust distribution.** The spatial distribution and the size distribution are input. DMS does not compute collisional or dynamical evolution, although sublimation, photosputtering and radiation-pressure blow-out modify the given distributions.

5. **Randomly oriented grains.** Spheres (Mie theory) or a distribution of hollow spheres (DHS). Grain alignment is not considered, so the thermal emission is unpolarised.

Section 12 lists further limitations.

1.3 What is new in version 2.2

Version 2.2 adds the loss of ice by photosputtering (Sect. 6.4): ultraviolet photons remove water molecules from icy grains, and a grain that loses its ice before it is destroyed by collisions, drifts inwards or reaches the age of the system becomes a grain of a residue component, as for sublimation. The test is made for every grain size and distance, optionally with the UV and P-R time of the orbit a grain follows after its release; in eccentric belts the ice is then decided per orbit. Without photosputtering the results are those of version 2.1. Version 2.2.1 adds the cookbook of Sect. 16: one recipe for every common task, with short scripts in `examples/recipes/`.

1.4 What is new in version 2.1

Version 2.1 generalises the temperature-dependent optical constants (Sect. 6.2): any number of components can use them at the same time, the data sets of a component are found from a base name and the temperatures in the file names, and n and k can be interpolated linearly in temperature (`tdep_mode = 'linear'`). Grains colder or hotter than all data sets of their component are reported. Input files of version 2.0 give the same results.

1.5 What is new in version 2.0

Version 2.0 is a substantial revision of the 2019 code. Table 1 lists the main changes; the complete list is in `CHANGELOG.md`.

Table 1: Main changes in DMS 2.0 with respect to the 2019 version.

Area	Change
Input	Namelist input file with keyword names and defaults; interactive questions kept
Geometry	Up to 10 belts, each with its own radial law, eccentricity and pericentre; Gaussian vertical profile; user-defined density function
Star	Model-atmosphere grids (PHOENIX, Castelli–Kurucz) with interpolation in $(T_{\rm eff}, \log g)$
Dust	Distribution of hollow spheres; composite grains from effective-medium theory (built-in rules and the `emc` code); grain sizes from a file
Optics	Mie calculation at the observing wavelengths with interpolated refractive index; energy-conserving treatment of the forward-scattering peak
Images	Q_ϕ, U_ϕ, degree of polarisation; PSF convolution; coronagraph mask; surface brightness units; refinement near the star
Python	Interface built with `numpy.f2py`; batch runner; tools for composite grains and stellar grids
Code	Library mode (errors are returned to the caller); one `Makefile`; new repository layout and documentation; automated test suite and continuous integration
Blow-out	Threshold for release at pericentre or apocentre of eccentric belts, or any value

1.6 Notation

Table 2 collects the symbols used in Part I. Wavelengths and grain radii are in micron in the input files, and all calculations are done in SI units.

Table 2: Symbols.

a	grain radius	$\mathcal{A}$	semi-major axis of a dust orbit
q	size-distribution exponent	$e,\ \omega$	eccentricity, argument of pericentre
r	3-d distance from the star	ϖ	distance in the disk plane
ψ	azimuth in the disk plane	Z	height above the mid-plane
i	inclination	PA	position angle
θ	scattering angle	ϕ	position angle of a pixel in the image
$C_{\rm abs},\ C_{\rm sca}$	cross sections	$Q_{\rm abs},\ Q_{\rm pr}$	efficiencies
$S_{11},\ S_{12}$	scattering-matrix elements	β	radiation pressure / gravity
$I_{\star,\lambda}$	stellar surface intensity	$B_\lambda(T)$	Planck function
$R_\star,\ M_\star,\ L_\star$	stellar radius, mass, luminosity	d	distance to the system
$\rho_{\rm b}$	bulk density of a grain	$n(\mathcal{A})$	radial density law (relative)
ρ	dust mass density	$\mu(r)$	surviving mass fraction
ε	dielectric function, $\varepsilon = m^2$	$m = n_{\rm r} + ik$	refractive index

2 Disk geometry

2.1 Coordinate frames

DMS uses three right-handed frames (Fig. 1).

- **Detector frame** $(x_{\rm d}, y_{\rm d}, z)$: $x_{\rm d}$ and $y_{\rm d}$ span the image; the observer is at $z \to +\infty$; the star is at the origin.
- **Sky frame** (x, y, z): the detector frame rotated by the position angle,

$$x = x_{\rm d}\cos{\rm PA} + y_{\rm d}\sin{\rm PA}, \qquad y = -x_{\rm d}\sin{\rm PA} + y_{\rm d}\cos{\rm PA}. \tag{1}$$

- **Disk frame** (X, Y, Z): Z is the disk normal and Y is the line of nodes, which is the projected major axis. With inclination i,

$$X = x\cos i - z\sin i, \qquad Y = y, \qquad Z = x\sin i + z\cos i. \tag{2}$$

The side $+X$ is the far side of the disk; the near side $(-X)$ faces the observer and shows the forward-scattering peak. For PA $= 0$ the major axis lies along the image y axis, as in earlier versions of DMS. A positive position angle rotates the major axis counter-clockwise. If the image is shown with $+y_{\rm d}$ to the north and east to the left, this is the astronomical position angle, counted from north through east.

2.2 Radial density laws

The disk consists of $N_{\rm b} \le 10$ belts. Belt k has a radial law $n_k(\mathcal{A})$, where $\mathcal{A}$ is the semi-major axis of the orbit through a point (for circular belts $\mathcal{A}$ is the distance from the star). The built-in laws are

$$\text{power law:} \quad n(\mathcal{A}) \propto (\mathcal{A}/{\rm au})^{-\alpha}, \tag{3}$$

$$\text{Ertel peak:} \quad n(\mathcal{A}) \propto s^\alpha \exp[\alpha(1-s)], \quad s = \frac{\mathcal{A} - R_{\rm start}}{R_{\rm peak} - R_{\rm start}}, \quad n = 0 \text{ for } \mathcal{A} < R_{\rm start}, \tag{4}$$

$$\text{Gaussian:} \quad n(\mathcal{A}) \propto (\mathcal{A}/{\rm au})^\alpha \exp\left[-\tfrac{1}{2}\left(\tfrac{\mathcal{A}-\mu}{\sigma}\right)^2\right], \tag{5}$$

$$\text{broken power law:} \quad n(\mathcal{A}) \propto \left[(\mathcal{A}/R_{\rm c})^{-2\alpha_{\rm in}} + (\mathcal{A}/R_{\rm c})^{-2\alpha_{\rm out}}\right]^{-1/2}, \tag{6}$$

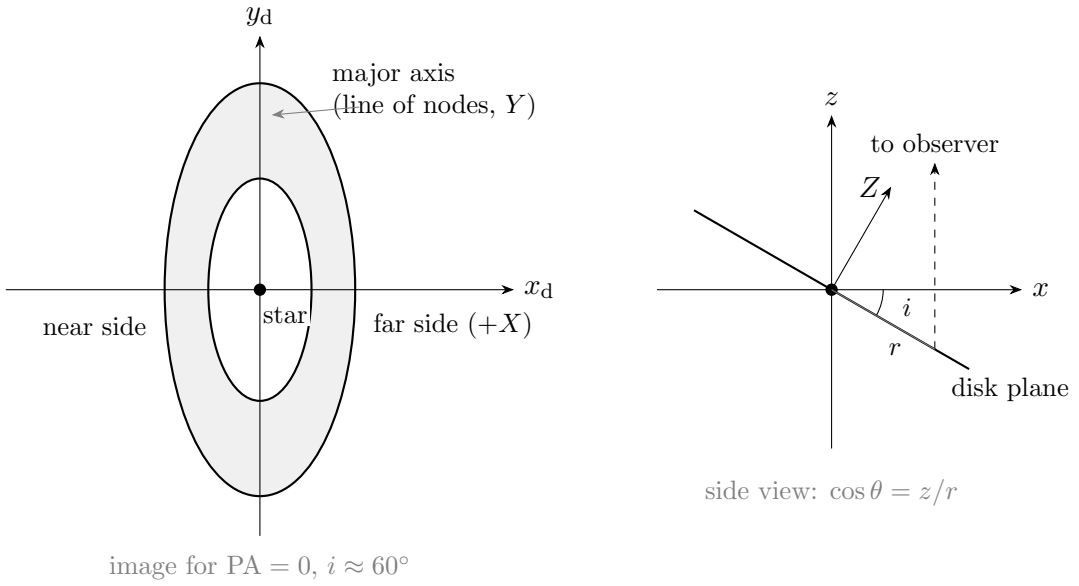


Figure 1: Left: the detector frame and the projected disk. Right: the disk plane seen from the side; the scattering angle θ of a grain is the angle between the direction from the star to the grain and the direction to the observer.

with $\alpha_{\rm in} > 0$ and $\alpha_{\rm out} < 0$ for the broken power law (Augereau et al. 1999). The law is evaluated only for $R_{\rm in} \le \mathcal{A} \le R_{\rm out}$; outside it the density is zero. A fifth law calls the function `user_density_fn` in `src/user_density.f90` (Sect. 23). It receives $\mathcal{A}$, the disk-frame coordinates and ten free parameters, so azimuthal clumps, arcs, spirals or gaps can be modelled without changing the rest of the code.

2.3 Eccentric belts

The orbits of a belt are confocal (the star sits in one focus), share the eccentricity e and are aligned, with the argument of pericentre ω measured in the disk plane from $+Y$ towards $+X$. A point at distance s from the star and azimuth $\psi = \mathrm{atan2}(X, Y)$ lies on the orbit with

$$\mathcal{A} = \frac{s\,[1 + e\cos(\psi - \omega)]}{1 - e^2}. \tag{7}$$

The particles are assumed to be spread uniformly in mean anomaly along each orbit. By Kepler's second law a particle spends a time $\propto s^2$ per unit true anomaly, so for a given $\mathcal{A}$ the number of particles per $\mathrm{d}\mathcal{A}\,\mathrm{d}\psi$ is proportional to s^2. At fixed azimuth, $\partial s/\partial\mathcal{A} = s/\mathcal{A}$, so the area element is $s\,\mathrm{d}s\,\mathrm{d}\psi = (s^2/\mathcal{A})\,\mathrm{d}\mathcal{A}\,\mathrm{d}\psi$. The factor s^2 cancels, and the surface density depends on $\mathcal{A}$ alone. The local thickness of the disk is proportional to the distance s for both vertical profiles below, so the volume density is

$$\rho(s, \psi) = n(\mathcal{A})\,\frac{\mathcal{A}}{s}, \tag{8}$$

which reduces to $n(s)$ for $e = 0$. The dust extends from the pericentre $R_{\rm in}(1 - e)$ to the apocentre $R_{\rm out}(1 + e)$. The grain temperatures follow the true distance. At a given $\mathcal{A}$ the surface density is the same at pericentre and apocentre and the dust at pericentre is warmer, so in a face-on image that resolves the width of the ring the surface brightness is highest at pericentre at all wavelengths (pericentre glow). The ring is wider at apocentre, by the factor $(1 + e)/(1 - e)$ for a belt with sharp edges, and therefore holds more dust per unit length along the ring. In the flux integrated across the ring this outweighs the lower temperature at millimetre wavelengths, where the emission depends only weakly on temperature: in the Rayleigh–Jeans limit the apocentre

is brighter by $[(1+e)/(1-e)]^{1/2} \approx 1+e$ (apocentre glow; Pan et al. 2016). The same holds in images whose beam is wider than the ring. For the images of `examples/eccentric_ring.nml` ($e = 0.3$, face-on) the apocentre-to-pericentre ratio is 1.27 for the 870 μm flux per unit length (summed over $\pm 5^\circ$ in azimuth around each apsis) and 0.69 for the peak surface brightness; at 24 μm the two ratios are 0.23 and 0.12.

2.4 Vertical structure

All belts share one vertical structure:

- **Wedge.** $\rho = n(\mathcal{A})\,\mathcal{A}/s$ inside $|Z| \le s \sin\vartheta$, where $s = r$ is the 3-d distance and ϑ the half opening angle; zero outside.
- **Gaussian.** $\rho = n(\mathcal{A})\,(\mathcal{A}/\varpi)\exp(-Z^2/2h^2)$ with $h = (h/r)\,\varpi$; here $s = \varpi$ is the distance in the disk plane and the radial law gives the mid-plane density. The profile is cut at $|Z| = c\,h$ ($c = 4$ by default).

2.5 Several belts

The amplitude of each belt is set so that belt k holds the fraction f_k of the dust mass that survives sublimation and blow-out (Sect. 6.3):

$$M_k = \int \rho_k(\mathbf{x})\,\mu(r)\,\mathrm{d}V, \qquad A_k \propto \frac{f_k}{M_k}, \tag{9}$$

where $\mu(r)$ is the surviving mass fraction at distance r. The integral is evaluated in spherical disk coordinates with 800 logarithmic radial cells, 48 latitude cells and 360 azimuth cells. Inclination, position angle and vertical structure are common to all belts.

3 The star

The stellar surface intensity $I_{\star,\lambda}$ comes from one of three sources:

1. a black body, $I_{\star,\lambda} = B_\lambda(T_{\rm eff})$;
2. an observed or model spectrum F_ν in Jy at the distance d, converted to $I_{\star,\lambda} = F_\nu\,(c/\lambda^2)\,d^2/(\pi R_\star^2)$;
3. a model-atmosphere grid: two columns, wavelength and surface flux $F_\lambda = \pi I_{\star,\lambda}$, for a set of $(T_{\rm eff}, \log g)$. DMS interpolates bilinearly in $(T_{\rm eff}, \log g)$ in $\log F_\lambda$ on the wavelength grid of the first model used.

Longward of the last tabulated wavelength of a spectrum, a Rayleigh–Jeans continuation is used; shortward of the first one the intensity is set to zero. The luminosity is $L_\star = 4\pi^2 R_\star^2 \int I_{\star,\lambda}\,\mathrm{d}\lambda$, and the flux at distance r from the star is $F_{\star,\lambda}(r) = \pi I_{\star,\lambda} R_\star^2/r^2$. The stellar mass enters only through the blow-out size.

4 Dust grains

4.1 Size distribution

The grain radii are binned logarithmically between $a_{\rm min}$ and $a_{\rm max}$. For the power law $\mathrm{d}N \propto a^{-q}\,\mathrm{d}a$, the weight of bin j with edges a_{j-1} and a_j is

$$w_j = \int_{a_{j-1}}^{a_j} a^{-q}\,\mathrm{d}a = \frac{a_j^{1-q} - a_{j-1}^{1-q}}{1-q}. \tag{10}$$

A single size and a table read from a file (radius and relative number per bin) are also possible. For a table, the bin edges are the geometric means of neighbouring radii. If there are several components with relative abundances X_c, all components share the same size distribution. X_c is therefore the fraction of grains by number, which is also the fraction by volume; the mass fractions follow from the bulk densities. The number of grains per kilogram of dust is

$$N_{jc} = \frac{X_c\, w_j}{\sum_{j',c'} X_{c'}\, w_{j'}\, m_{j'c'}}, \qquad m_{jc} = \tfrac{4}{3}\pi a_j^3\, \rho_{\mathrm{b},c}. \tag{11}$$

4.2 Composite grains: effective-medium theory

A grain made of several materials can be described by an effective dielectric function $\varepsilon_{\mathrm{eff}}$ (with $\varepsilon = m^2$) and then treated as a homogeneous sphere. DMS has four rules built in, for an inclusion with volume fraction f and dielectric function ε_i in a matrix ε_m:

$$\text{Maxwell Garnett:} \quad \varepsilon_{\mathrm{eff}} = \varepsilon_m \frac{\varepsilon_i + 2\varepsilon_m + 2f(\varepsilon_i - \varepsilon_m)}{\varepsilon_i + 2\varepsilon_m - f(\varepsilon_i - \varepsilon_m)}, \tag{12}$$

$$\text{Bruggeman:} \quad f\,\frac{\varepsilon_i - \varepsilon_{\mathrm{eff}}}{\varepsilon_i + 2\varepsilon_{\mathrm{eff}}} + (1-f)\,\frac{\varepsilon_m - \varepsilon_{\mathrm{eff}}}{\varepsilon_m + 2\varepsilon_{\mathrm{eff}}} = 0, \tag{13}$$

$$\text{Lichtenecker:} \quad \ln \varepsilon_{\mathrm{eff}} = f \ln \varepsilon_i + (1-f) \ln \varepsilon_m, \tag{14}$$

$$\text{Looyenga:} \quad \varepsilon_{\mathrm{eff}}^{1/3} = f\, \varepsilon_i^{1/3} + (1-f)\, \varepsilon_m^{1/3}. \tag{15}$$

For more than two materials, non-spherical inclusions (needles, platelets) and coated grains, the Python module `dms.emc` drives the code `emc` of Ossenkopf (1991), which is not part of DMS and must be installed separately (Sect. 20.2). The combinations used in the water-ice studies are listed in Table 3.

Table 3: Combinations of `emc` theories and inclusion shapes used in our water-ice studies. `dms.emc` accepts any combination of the theories and shapes listed in Sect. 20.2.

Label	Theory	Shape of the inclusions
MG sphere, MG needle, MG platelet	Maxwell Garnett	sphere, needle, platelet
BG sphere	Bruggeman	sphere
BG needle, BG platelet	Bruggeman	needle or platelet (ice), sphere (refractory)
UniMG coated, UniBG coated	universal MG, universal BG	refractory core coated with ice

All four rules assume inclusions much smaller than the wavelength. Maxwell Garnett treats matrix and inclusion differently and is meant for dilute inclusions; Bruggeman treats all components alike.

Porosity. Porous grains are mixtures with vacuum ($\varepsilon = 1$) as one component.

Internal and external mixing. In an internal mixture each grain contains all materials and has one effective dielectric function. In an external mixture each material forms its own grains; DMS treats every material as a separate component with its own abundance, temperature and blow-out size.

Bulk density. The bulk density of every component is an input. For an internal mixture it is the volume-weighted mean, $\rho_{\mathrm{b}} = \sum_j f_j \rho_{\mathrm{b},j}$, with vacuum counted as zero; DMS does not compute it.

4.3 Distribution of hollow spheres

Irregular grains can be approximated by a distribution of hollow spheres (DHS; Min et al. 2005). The vacuum volume fraction f of the central cavity is uniformly distributed between 0 and $f_{\max}$. The material volume, and therefore the mass, is that of a solid sphere of radius a; the outer and inner radii are

$$a_{\rm out} = a\,(1-f)^{-1/3}, \qquad a_{\rm core} = a\,(1-f)^{-1/3} f^{1/3}. \tag{16}$$

Cross sections and scattering-matrix elements are averaged over f with n_f Gauss–Legendre points; S_{11} and S_{12} are weighted with $C_{\rm sca}$. Efficiencies refer to πa^2. The coated-sphere coefficients are computed with the algorithm of Yang (2003), as used by Peña & Pal (2009). The downward recurrences start at $1.07\max(n_{\max}, |mx|) + 30$; with a margin of only about 16 terms, large and nearly non-absorbing spheres were wrong by up to 0.1 per cent. Min et al. (2005) took $f_{\max} = 1$, the most extreme shape distribution, as their default and compared $f_{\max} = 0.3$–0.5 with laboratory measurements of quartz; Min et al. (2016) adopted $f_{\max} = 0.8$. DMS accepts $0 < f_{\max} \le 1$ (default 0.8) and $1 \le n_f \le 100$ (default 10); with $n_f = 10$ and $f_{\max} = 1$ the largest node is $f = 0.987$.

One coated-sphere calculation takes about 4–5 times as long as a Mie calculation of the same size parameter, and the DHS needs n_f of them at outer size parameters up to $x\,(1-f_{\max})^{-1/3}$, so that the DHS costs about 50 times the Mie calculation of compact spheres. Both grow in proportion to x, and the large grains at short wavelengths dominate: for grains up to 1 mm and optical data from 0.1 μm, 90 per cent of the time goes into $x > 1000$. In geometrical optics the efficiencies of a large grain depend on its size only through the absorption along the paths inside it, i.e. through kx. DMS therefore computes a grain with $x > x_{\max}$ (`dhs_xmax`, default 1000) at $x_{\max}$ with $k' = k\,x/x_{\max}$, which keeps kx; beyond $kx = 30$ the grain is opaque, and k' is limited to $30/x_{\max}$ (or k is kept if $k\,x_{\max} \ge 30$), so that $k' \ll n$ and the reflection at the surface is unchanged. For $x = 3\times10^3$–7×10^4 the efficiencies agree with the full DHS to better than 1 per cent for absorbing material ($k \gtrsim 0.01$); for nearly transparent material ($k \sim 10^{-7}$–10^{-5}, $Q_{\rm abs} < 10^{-3}$) $Q_{\rm abs}$ differs by up to 10 per cent, as much as the full DHS changes between 10 and 40 points in f. In a debris ring around an A star with forsterite and astronomical silicate (grains 0.26–1000 μm) the grain temperatures change by less than 0.15 per cent and the fluxes at 33–100 μm by less than 0.05 per cent (0.4 per cent at 10 μm), while the calculation is five times faster. The approximation is used on the internal wavelength grid and, without scattered light, at the observing wavelengths; with scattered light the observing wavelengths use the full DHS. `dhs_xmax = 0` gives the full DHS for all sizes.

5 Optical properties

5.1 Mie theory and wavelength grids

Optical data files give $n_{\rm r}$ and k against wavelength. DMS uses two wavelength grids.

- The *internal grid* is the union of the wavelengths of all optical-data files within the range that all files cover; every data set is interpolated onto it, linearly in $\log\lambda$. The order of the components therefore does not matter. On this grid DMS computes $Q_{\rm abs}$ and $Q_{\rm pr}$, which enter the temperatures and the radiation pressure.
- The *observing wavelengths* are given by the user. At these wavelengths DMS repeats the Mie calculation with the refractive index interpolated to each observing wavelength, which is more accurate than interpolating the efficiencies, in particular inside absorption bands.

The Mie routine is `miex` (Wolf & Voshchinnikov 2004), which was written for very large size parameters. For DMS 2.0 its work array is allocated with the number of terms actually needed, and it keeps no saved variables, so it can be called from OpenMP threads.

5.2 Scattering matrix

For every grain size, material and observing wavelength, DMS stores $C_{\rm abs}$, $C_{\rm sca}$ and the normalised matrix elements

$$\hat{S}_{11}(\theta) = \frac{S_{11}(\theta)}{k^2 C_{\rm sca}}, \qquad \hat{S}_{12}(\theta) = \frac{S_{12}(\theta)}{k^2 C_{\rm sca}}, \qquad \int_{4\pi} \hat{S}_{11}\,\mathrm{d}\Omega = 1, \tag{17}$$

with $k = 2\pi/\lambda$ (Bohren & Huffman 1983), on N_θ equidistant angles between 0° and 180° (181 by default). The degree of linear polarisation of light scattered by one grain is $-S_{12}/S_{11}$; it is positive for polarisation perpendicular to the scattering plane, where $S_{12} < 0$.

Forward-scattering peak. DMS interpolates $\hat{S}_{11}$ and $\hat{S}_{12}$ linearly in θ between the grid angles. For large grains the forward diffraction peak (width $\approx 1/x$ rad, $x = 2\pi a/\lambda$) and the first diffraction rings are much narrower than the angle step, and interpolating point values there would put far too much light at small scattering angles, i.e. into edge-on disks and lines of sight close to the star. DMS therefore

1. averages $\hat{S}_{11}$ and $\hat{S}_{12}$ over the angle bin of each grid angle in the forward region ($\theta \lesssim 30/x$), computed from the Mie coefficients with at least four samples per ring period;
2. normalises $\int \hat{S}_{11}\,\mathrm{d}\Omega = 1$ with the exact weights of the linear interpolation used in the integration, by rescaling $\hat{S}_{11}$ and $\hat{S}_{12}$ only in the forward region $\theta < 6/x$ (at least the first three grid angles), where the energy missed by the grid lies; for small grains and Henyey–Greenstein functions this region is the whole sphere;
3. refines volume elements (images) and integration cells (SED) close to the forward direction (Sect. 8).

Beyond the forward region the values at the grid angles are used. Section 11 gives the accuracy reached.

5.3 Henyey–Greenstein phase functions

Instead of Mie theory, the phase function can be a one- or two-component Henyey–Greenstein function,

$$p(\theta) = w\,\mathrm{HG}(g_1,\theta) + (1-w)\,\mathrm{HG}(g_2,\theta), \qquad \mathrm{HG}(g,\theta) = \frac{1}{4\pi}\frac{1-g^2}{(1+g^2-2g\cos\theta)^{3/2}}, \tag{18}$$

with a Rayleigh-like degree of polarisation

$$P(\theta) = -p_{\max}\frac{\sin^2\theta}{1+\cos^2\theta}, \qquad \hat{S}_{12} = P\,p(\theta). \tag{19}$$

This Rayleigh-like dependence of the polarisation on the scattering angle was used, for example, by Engler et al. (2017). The cross sections $C_{\rm abs}$ and $C_{\rm sca}$ still come from Mie theory.

5.4 Radiation pressure and blow-out

The ratio of radiation pressure to gravity is (Burns et al. 1979)

$$\beta(a) = \frac{3L_\star\,\langle Q_{\rm pr}\rangle_\star}{16\pi G M_\star c\,\rho_{\rm b}\,a}, \qquad \langle Q_{\rm pr}\rangle_\star = \frac{\int Q_{\rm pr}(a,\lambda)\,I_{\star,\lambda}\,\mathrm{d}\lambda}{\int I_{\star,\lambda}\,\mathrm{d}\lambda}, \tag{20}$$

with $Q_{\rm pr} = Q_{\rm ext} - g\,Q_{\rm sca}$, where $g = \langle\cos\theta\rangle$ is the asymmetry parameter.

A grain released at distance r from a parent body on an orbit with semi-major axis $\mathcal{A}$ has the parent's velocity, but feels the stellar gravity reduced by the factor $1-\beta$. Its orbit is unbound if

$$\beta \geq \beta_{\mathrm{crit}} = \frac{r}{2\mathcal{A}} = \frac{1-e^2}{2\,(1+e\cos f)}, \tag{21}$$

where f is the true anomaly of the release point: $\beta_{\mathrm{crit}} = 1/2$ for circular orbits, $(1-e)/2$ for release at pericentre and $(1+e)/2$ for release at apocentre. With `blowout = .true.`, grains with $\beta \geq \beta_{\mathrm{crit}}$ are removed. The input `blowout_release` selects β_{crit}: `'circular'` (1/2, the default), `'pericentre'` or `'apocentre'`, using the eccentricity of each belt; `blowout_beta` sets any other value. The population of grains in DMS depends on the distance from the star, not on the azimuth, so β_{crit} is applied per distance: within the distance range of a belt its value is used, where belts overlap the smaller value applies, and the smallest and largest value are reported (summary, FITS keywords `BETACMIN` and `BETACMAX`, `Result.blowout_beta`). Because $\langle Q_{\mathrm{pr}}\rangle_\star$ depends on the composition and on the stellar spectrum, the blow-out size differs between materials and stars. It is computed for every component, after sublimation and with the temperature-dependent optical data if these are used.

6 Temperatures and sublimation

6.1 Radiative equilibrium

A grain of radius a at distance r absorbs $\int C_{\mathrm{abs}}\,\pi I_{\star,\lambda} R_\star^2/r^2\,\mathrm{d}\lambda$ and emits $4\pi\int C_{\mathrm{abs}} B_\lambda(T)\,\mathrm{d}\lambda$. Equating the two gives the distance at which a grain has temperature T:

$$r(T) = \frac{R_\star}{2}\left[\frac{\int C_{\mathrm{abs}}(a,\lambda)\,I_{\star,\lambda}\,\mathrm{d}\lambda}{\int C_{\mathrm{abs}}(a,\lambda)\,B_\lambda(T)\,\mathrm{d}\lambda}\right]^{1/2}. \tag{22}$$

For a black-body grain around a black-body star this gives the familiar $T = T_{\mathrm{eff}}\,(R_\star/2r)^{1/2}$. DMS evaluates Eq. (22) on a table of temperatures for every grain size and material, makes $r(T)$ monotonic, and interpolates $\log r$ to obtain $T(r)$ on a logarithmic distance grid that covers all dust. The factor $R_\star^2/4r^2$ assumes that the star appears as a point; within a few stellar radii the dilution factor of an extended star would be needed.

6.2 Temperature-dependent optical constants

Optical data can be given at several temperatures for any number of components (e.g. crystalline water ice and the three crystallographic axes of forsterite, each treated as a component). For such a component (`tdep = .true.`) `optical_data` is a base name, and DMS uses every file `<base>_<T>K` or `<base>_<T>K.<extension>` in the optical-data directory, e.g. `composite_cice_010K.txt`, `composite_cice_015K.txt`, … for the base name `kim2019/pure_ices/composite_cice`. The temperature is read from the file name (integer or decimal, leading zeros allowed) and the data sets are sorted by it; two files with the same temperature are an error. The directory is listed with the POSIX routines `opendir` and `readdir` (a small C function called through `ISO_C_BINDING`).

Each grain uses data at its own equilibrium temperature T, which in turn depends on the data; the choice is made self-consistently in the temperature calculation (Eq. 22 is evaluated with the data belonging to each temperature of the table) and then applies to the radiation pressure, sublimation, emission and scattering of that grain, for every grain size, component and distance. With `tdep_mode = 'nearest'` (the default, as in version 2.0) a grain uses the data set whose temperature is closest to T. With `tdep_mode = 'linear'`, DMS first adds data sets between each pair of neighbouring sets $T_1 < T_2$, with

$$(n,k)(T) = (1-w)\,(n,k)(T_1) + w\,(n,k)(T_2), \qquad w = \frac{T-T_1}{T_2-T_1}, \tag{23}$$

spaced by at most `tdep_dt` (5 K by default); a grain then uses the closest of these, so its n and k are interpolated to within `tdep_dt`/2 of its temperature. Every data set is one more Mie calculation, which sets the cost of a small `tdep_dt`; the optics cache keeps them. With scattered light the scattering matrix is stored for every data set, so its memory grows in the same way (DMS prints it when it exceeds 1 GB); for thermal emission alone it is not needed. Interpolating n and k does not move a band whose position changes with temperature: between two sets with bands at different wavelengths it gives two weaker bands. Where the band position matters, data sets closer in temperature are needed. Grains colder (hotter) than all data sets of their component use the coldest (hottest) set; DMS reports the temperature range of these grains once per component on the screen and in the summary.

6.3 Sublimation

Each component can have a sublimation temperature T_s. Where a grain is hotter than T_s, it either disappears or turns into another component (the *residue*); for example, ice–silicate grains become porous silicate grains (Kim et al. 2019). The components are processed in order of increasing T_s, so a residue can itself sublimate further in. Photosputtering (Sect. 6.4) is applied before sublimation and blow-out after it. Photosputtering, sublimation and blow-out together define, at every distance, the number of grains of every size and component per kilogram of initial dust and the surviving mass fraction $\mu(r)$. The input dust mass refers to the surviving grains.

6.4 Ice loss by photosputtering

Ultraviolet photons absorbed by water ice remove molecules from its surface with a yield Y of about 10^{-3} molecules per photon (Grigorieva et al. 2007, who adopt this value between the estimates of Westley et al. 1995, from the laboratory, and Andersson et al. 2006, from molecular dynamics). Following Grigorieva et al. (2007, their eq. 7 with the whole surface icy), the ice front of a grain of radius a recedes at $m_{\mathrm{H_2O}} Y N_\mathrm{abs}/(4\rho_\mathrm{w})$, so its ice is gone after

$$t_\mathrm{ps} = \frac{4\rho_\mathrm{w}\, a}{m_{\mathrm{H_2O}}\, Y\, N_\mathrm{abs}}, \qquad N_\mathrm{abs}(r) = \int_{\lambda_1}^{\lambda_2} Q(\lambda)\, \frac{\pi I_{\star,\lambda} R_\star^2}{r^2}\, \frac{\lambda}{hc}\, \mathrm{d}\lambda + \bar{Q}\, N_\mathrm{ISRF}, \tag{24}$$

with ρ_w the ice mass per grain volume (`ps_ice_fraction` times the bulk density), λ_1–λ_2 the band 0.0912–0.2431 μm(13.6–5.1 eV), Q the absorption efficiency of the grain or 1, and N_ISRF an optional interstellar photon flux ($\bar{Q}$: band mean of Q). The stellar intensity can be replaced in the band by a measured UV spectrum (`ps_uv_file`); this matters for K and M stars, whose chromospheric UV is missing from photospheric models. A grain loses its ice where t_ps is shorter than a reference time: the collisional lifetime (Grigorieva et al. 2007, eqs. 19–20)

$$t_\mathrm{coll} = \frac{1}{\tau\Omega}\left[\left(\frac{a}{s_1}\right)^{-2} + \left(\frac{a}{s_2}\right)^{2.7}\right] \ (a < s_2), \qquad t_\mathrm{coll} = \frac{1}{\tau\Omega}\left(\frac{a}{s_2}\right)^{0.3} \ (a \geq s_2), \tag{25}$$

with $s_1 = 1.2\, s_{1/2}$, $s_2 = 100\, s_{1/2}$, $s_{1/2}$ the largest radius with $\beta \geq 1/2$ at that distance (the smallest grain if no grain reaches 1/2), the vertical optical depth τ and the Kepler frequency Ω; the Poynting–Robertson drag time $t_\mathrm{PR} = r^2 c/(2\beta G M_\star)$ (their eq. 16); the age of the system; or the shortest of these. Since t_ps and t_PR both grow as r^2, a grain size loses its ice everywhere or nowhere when only P-R drag is considered; against collisions ($t_\mathrm{coll} \propto r^{3/2}$) and against the age the ice is lost inside a radius that decreases with the grain size. The grain then becomes a grain of the residue component (`ps_residue`) or vanishes. DMS lists the largest distance with ice loss for every grain size in the summary and in Python (`res.photosputtering`).

With `ps_orbit = 'release'` the stellar photons are averaged over the Kepler orbit a grain follows after its release at r from its parent orbit $(a_\mathrm{p}, e_\mathrm{p})$, with $a_\mathrm{p} = r$, $r/(1-e)$ or $r/(1+e)$ for

release from circular orbits, at pericentre or at apocentre of the belt (`blowout_release`, Sect. 5.4):

$$\frac{1}{a_\mathrm{g}} = \frac{1/a_\mathrm{p} - 2\beta/r}{1-\beta}, \qquad 1 - e_\mathrm{g}^2 = \frac{a_\mathrm{p}(1-e_\mathrm{p}^2)}{(1-\beta)\,a_\mathrm{g}}, \qquad \langle r^{-2}\rangle = \frac{1}{a_\mathrm{g}^2\sqrt{1-e_\mathrm{g}^2}}. \tag{26}$$

N_abs is scaled by $r^2\langle r^{-2}\rangle$, which is $(1-2\beta)^{3/2}/(1-\beta)$ for circular parent orbits (0.80, 0.58 and 0.15 at $\beta = 0.1$, 0.2 and 0.4), and the P-R time is that of the eccentric orbit, $t_\mathrm{PR} = a_\mathrm{g}^2 c\,(1-e_\mathrm{g}^2)^{3/2}/[\beta G M_\star(2+3e_\mathrm{g}^2)]$ from the orbit-averaged decay of a (Wyatt & Whipple 1950). The collisional lifetime and the age are not changed, and the grains stay at their release distance.

In an eccentric belt the dust at one distance from the star belongs to different orbits, and the grains of one orbit pass through its inner and outer parts. With `ps_orbit = 'release'` the ice is therefore decided per orbit: every eccentric belt is divided into `ps_orbit_bins` bins in semi-major axis a (logarithmic, default 32), and for every bin, size and component the test above is made with the parent orbit (a, e) of the belt, released at its pericentre, at its apocentre or from a circular orbit (`blowout_release`), and with the Kepler frequency at a for the collisional lifetime. A point of the belt takes the population of the orbit through it (`belt_point` gives a of the point); temperatures, sublimation and blow-out remain functions of the distance. The kernels (thermal emission, cross section, mass and f_d) of every pair of orbit bin and distance are stored after the distance entries (`src/orbit_populations.f90`); scattering takes the grains of the orbit bin directly. With no ice lost the orbit bins reproduce the run without photosputtering to 10^{-10}, and for $e \to 0$ the circular belt, where the ice is decided per distance.

This is a threshold model on fixed orbits. Erosion during the lifetime of a grain, the growth of β as the grain shrinks, the resulting outward drift (Grigorieva et al. 2007) and the spreading of the released grains over their orbits are not followed.

7 Emission, scattering and polarisation

7.1 Contribution of one volume element

Consider a volume element $\mathrm{d}V$ at position $\mathbf{x}$, distance r from the star, containing the dust mass $\mathrm{d}m = \rho\,\mathrm{d}V$. With $N_{jc}(r)$ grains of size j and component c per unit mass, the flux densities received at distance d are

$$\mathrm{d}F_\lambda^\mathrm{th} = \frac{\mathrm{d}m}{d^2}\sum_{j,c} N_{jc}\,C_{jc}^\mathrm{abs}(\lambda)\,B_\lambda(T_{jc}(r))\,, \tag{27}$$

$$\mathrm{d}F_\lambda^\mathrm{sca} = \frac{\mathrm{d}m}{d^2}\sum_{j,c} N_{jc}\,C_{jc}^\mathrm{sca}(\lambda)\,\hat{S}_{11,jc}(\lambda,\theta)\,\frac{\pi I_{\star,\lambda} R_\star^2}{r^2}, \tag{28}$$

$$\mathrm{d}P_\lambda = \frac{\mathrm{d}m}{d^2}\sum_{j,c} N_{jc}\,C_{jc}^\mathrm{sca}(\lambda)\,\hat{S}_{12,jc}(\lambda,\theta)\,\frac{\pi I_{\star,\lambda} R_\star^2}{r^2}, \tag{29}$$

with the scattering angle $\cos\theta = z/r$ (observer at $+z$). The sums are tabulated before the integration as kernels per kilogram of dust at every distance of the distance grid, and $\hat{S}_{11}$, $\hat{S}_{12}$ are interpolated linearly in θ. Flux densities per unit wavelength are converted to Jy with $F_\nu = F_\lambda\lambda^2/c$.

7.2 Stokes parameters

$\mathrm{d}P_\lambda < 0$ means polarisation perpendicular to the scattering plane. With $\phi = \mathrm{atan2}(y_\mathrm{d}, x_\mathrm{d})$, the Stokes parameters are

$$\mathrm{d}Q = \mathrm{d}P\cos 2\phi, \qquad \mathrm{d}U = \mathrm{d}P\sin 2\phi, \tag{30}$$

so that $Q > 0$ means polarisation along the image x axis. The polarised intensity is $\mathrm{PI} = \sqrt{Q^2 + U^2}$, and the azimuthal Stokes parameters are

$$Q_\phi = -(Q\cos 2\phi + U \sin 2\phi), \qquad U_\phi = Q \sin 2\phi - U \cos 2\phi. \tag{31}$$

For optically thin single scattering the polarisation is perpendicular to the direction to the star, so $Q_\phi > 0$ and $U_\phi \approx 0$. Observers often count ϕ from north through east; the sign of Q_ϕ is the same in that convention, while U_ϕ may change sign. The degree of polarisation is PI/I, where I is the dust intensity (thermal plus scattered, without the star).

7.3 Global quantities

- **Fractional luminosity:** $f_\mathrm{d} = \sum N\pi a^2 \langle Q_\mathrm{abs}\rangle_\star/(4\pi r^2)$, summed over all grains. It is accumulated during the integration. The integrated thermal SED reproduces it to better than 0.1 per cent (Sect. 11).
- **Geometrical optical depth** ($Q_\mathrm{ext} = 1$): along every line of sight (`image_tau`); vertically, $\tau_\perp(\varpi) = \int n\pi a^2 \,\mathrm{d}Z$, as an azimuthal mean and towards pericentre and apocentre; and as the mean $\tau_\mathrm{mean} = \sum N\pi a^2/S$, with the disk area $S = \sum_k \pi(R^2_{\mathrm{out},k} - R^2_{\mathrm{in},k})\sqrt{1-e_k^2}$.
- **Mass scaling:** all quantities are first computed per unit density and then scaled so that the surviving dust mass equals the input dust mass.

8 Images and adaptive volume refinement

8.1 Image grid

The image has $N \times N$ pixels (N odd), with the star in the central pixel. The field covers $\pm R_\mathrm{field}$, where R_field is the largest distance of dust from the star: the apocentre of the outermost orbit for the wedge, and that distance divided by $\cos\vartheta_\mathrm{cut}$ for the Gaussian profile, whose cut extends above the disk plane. Along the line of sight through each pixel, the model space is divided into cubes of the pixel size. Each cube is a *volume element*; its contribution to the pixel is computed with Eqs. (27)–(29) at its centre.

8.2 Why refinement is needed

A volume element of one pixel can be much larger than the scales on which the integrand varies: the disk edges, a thin disk seen edge-on, a narrow belt, or the steep temperature gradient close to the star. Evaluating the integrand once per element would then misplace or miss flux. A finer image grid everywhere would be expensive, since most elements contain either no dust or smoothly varying dust. DMS therefore refines only where needed.

8.3 Splitting criteria

A volume element (cube of half size h) is split into eight sub-cubes of half size $h/2$, and the test is repeated for each of them (an octree, Fig. 2). A cube is *not* split further if all of the following hold:

1. **Temperature.** The temperature of the smallest grain of the first component, evaluated at the smallest and the largest distance of the cube from the star, differs by less than ΔT_max (`max_delta_t`, 2 K by default).

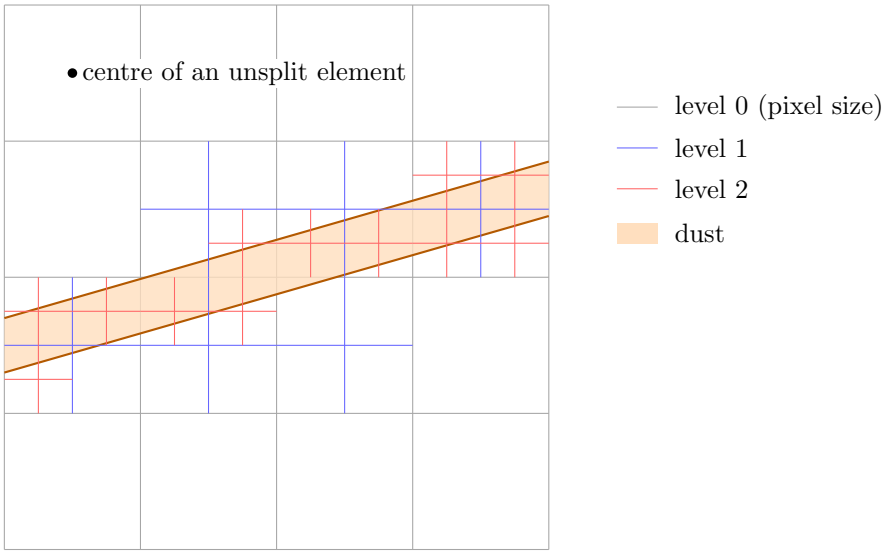


Figure 2: Adaptive refinement, shown in two dimensions (the code splits cubes into eight sub-cubes). Elements crossing the edges of the dust distribution or a strong temperature gradient are split; elements inside smooth dust or in empty space are evaluated once.

2. **Density.** The density at the eight corners and at the centre is either non-zero everywhere and smooth, $|(\rho_{\max} + \rho_{\min})/2 - \rho_{\mathrm{c}}| \leq 0.01\,\rho_{\mathrm{c}}$, or zero everywhere. A cube that is zero at all nine points but thicker than the local disk thickness is still split, so that a thin disk cannot slip between the test points.

The splitting level is limited by $N_{\max}$ (`n_split_max`, 3 by default), so that a pixel-sized element is divided into at most $8^{N_{\max}}$ sub-cubes.

Refinement near the star. Close to the star a volume element can be large compared with its distance to the star, for example when dust extends to 0.01 au in an image of ±150 au. With the cap $N_{\max}$ alone, the warm inner dust is then under-sampled. In DMS 2.0 the splitting continues beyond $N_{\max}$ (temperature and boundary criteria only) as long as the cube is larger than 0.05 times its distance to the star, $2h > 0.05\max(r_{\min}, R_{\mathrm{in,eff}})$, up to an absolute limit of 30 levels. Elsewhere $N_{\max}$ applies as before.

Refinement near the forward direction and in narrow belts. When scattered light is computed, elements whose scattering angle lies within two angle-grid steps of the forward direction are split, also beyond $N_{\max}$, until their angular size seen from the star is below a quarter of the grid step and they are smaller than half the width of the narrowest belt. In addition, an element that overlaps a belt and has zero density at all nine test points is split while it is wider than half the belt width, so that a belt narrower than an element cannot fall between the test points.

8.4 Implementation

- **Depth-first stack.** The sub-cubes are kept on a stack (at most seven pending entries per level), so the memory needed per thread is small and fixed.
- **Rejection of empty space.** Elements whose distance range lies entirely inside the innermost pericentre or outside the largest dust distance are skipped without evaluation.

- **Parallelisation.** Pixels are distributed over OpenMP threads; the routines called in the inner loop only read global data.
- **Mirror symmetry.** If PA = 0 and every belt is symmetric under $Y \to -Y$ (circular, or eccentric with the pericentre on the minor axis; user densities only when declared symmetric), only half of the image is computed and mirrored.
- **Diagnostics.** Before the integration, DMS prints the temperature difference of the smallest grains across the finest sub-volume at the inner edge, $T(R_{\rm in,eff}) - T(R_{\rm in,eff} + \sqrt{3}\,\Delta/2^{N_{\rm max}})$, where Δ is the pixel size, and warns if it exceeds $\Delta T_{\rm max}$ (`max_temp_diff_cals`).

9 SEDs

DMS obtains SEDs in two ways.

1. **Direct quadrature** (`mode = 'sed'`). The disk is integrated in spherical disk coordinates: 400 logarithmic radial cells between the innermost pericentre and the largest dust distance, 16 latitude cells within the opening angle for the wedge (48 for the Gaussian profile) and 180 azimuth cells, with additional refinement near the forward direction. Every cell centre is transformed to the detector frame, so the scattering angles and the polarisation are those of the actual inclination. Equations (27)–(29) are evaluated at the cell centres. This is fast and needs no image.
2. **Summed images** (`mode = 'both'`). The images are computed and their pixels summed. This is the natural choice when images are needed anyway.

For the thermal SED of the test models the two methods agree within 0.07 per cent (Sect. 11). The scattered-light and polarised SEDs depend on the inclination through the phase function; the thermal SED does not.

10 Simulated observations

If a PSF or a mask is requested, DMS writes a second set of images, `image_*_obs`.

- **PSF.** A normalised 2-d Gaussian truncated at 4σ, with $\sigma = \mathrm{FWHM}/(2\sqrt{2\ln 2})$. The FWHM is given per wavelength, or set by diffraction, $\mathrm{FWHM} = 1.029\,\lambda/D$ for a telescope of diameter D. The convolution is separable and conserves flux, apart from flux leaving the field.
- **Polarisation after convolution.** Q and U are convolved; PI, Q_ϕ, U_ϕ and the degree of polarisation are then derived from the convolved Q and U, as in an observation.
- **Star.** With `add_star`, the star sits in the central pixel of the total image before the convolution, so `image_total_obs` contains the stellar PSF.
- **Mask.** Pixels within the mask radius are set to NaN. The mask has a sharp edge; its transmission profile and the coronagraphic suppression of the stellar halo are not modelled.
- **Noise.** No noise is added; the images are noise-free model images.
- **Units.** Images are kept in Jy pixel^{-1} internally and written in Jy pixel^{-1}, mJy arcsec^{-2} or Jy arcsec^{-2}.

11 Verification

Table 4 summarises the tests made for version 2.0. All use the astronomical silicate of Draine (2003) unless noted otherwise. The checks that run in about a minute are part of the automated test suite (Sect. 13.4); the others were made once for this release.

Table 4: Verification tests of DMS 2.0.

Test	Result
Energy balance	Two eccentric belts, Gaussian profile: integrated thermal SED$/L_\star$ = 1.0006 f_d
Image SED vs. direct quadrature	Two belts, Gaussian profile: agreement within 0.07 per cent
Belt mass fractions	Input 5/95 per cent; recovered 4.97/95.03 per cent from $\tau_\perp$
Size distribution from a file	Identical to the same power law (f_d and SED to 10^{-6})
Model-atmosphere grid	Black-body grid: identical to the black-body star on a grid node; 0.7 per cent difference in $L_\star$ between the 5500 and 6000 K nodes, from the interpolation in $\log F_\lambda$
PSF convolution	Gaussian PSF (FWHM 0.5″) and a 0.3″ mask, single ring: 0.088750 vs. 0.088755 Jy at 70 μm
Refinement near the star	Original 2019 test model (dust from 0.01 au, 101 pixels, N_max = 3): thermal flux at 1/10/100 μm before the fix 1.1×10^{-30}/0.20/7.44 Jy, after 1.39/12.5/8.03 Jy; direct quadrature 1.37/12.7/8.06 Jy
DHS, $f_\mathrm{max} \to 0$	Equal to compact Mie spheres within 10^{-5}
Coated spheres	Agree with `scattnlay` (Peña & Pal 2009) to 10^{-10} in Q and 10^{-7} in S_{11}, S_{12} for size parameters 0.5–10^4
Forward scattering	Exact integration of the Mie phase function (single size, ring at 100 au, 5° opening angle, 1.6 μm, x = 1–3927, i = 60° and 88°): SED within 1.5 per cent (largest deviation for $x \approx 40$ edge-on), images within 0.8 per cent for rings 2 au wide or wider, within 6 per cent for a 0.1-au ring seen edge-on
Polarisation	Stokes Q, U of an inclined, rotated ring within 1 per cent of the analytic result
Several optical-data grids	External ice mixtures with astrosilicate, pyroxene and organics against an independent Mie code: within 0.5 percentage points in the degree of polarisation
Threads	1 and 2 OpenMP threads differ by 2×10^{-13}
Robustness	31 input cases (edge cases and invalid input) under AddressSanitizer, UndefinedBehaviorSanitizer, `-fcheck=all` and floating-point traps: no memory errors or undefined behaviour
Input routes	Namelist, interactive questions and Python give identical results
Compilers	`gfortran` 10.5 and 13.3; `make debug` without warnings

12 Limitations

Besides the assumptions in Sect. 1.2, the following points limit the use of DMS.

- **Blow-out in eccentric belts.** In a belt with eccentricity e the blow-out threshold depends on where along the orbit a grain is released, between $(1 - e)/2$ and $(1 + e)/2$ (Eq. 21). DMS applies one threshold per distance (`blowout_release`, `blowout_beta`), not one that varies with the azimuth, and it does not follow the new orbits of bound grains released with $\beta < \beta_\mathrm{crit}$.
- **Photosputtering.** A threshold model on fixed Kepler orbits (Sect. 6.4): the erosion of a

grain during its lifetime, the growth of β as it shrinks and the resulting drift are not followed, and P-R drag enters only as a time scale. The release point on the parent orbit is the one of `blowout_release`; an average over release points along the orbit is not available. The collisional lifetime uses one vertical optical depth (`ps_tau`) for the whole disk. Stellar models lack the chromospheric UV of cool stars (`ps_uv_file`).

- **One size distribution.** All belts share one size distribution, which changes with distance only through sublimation, photosputtering and blow-out. A component whose small and large grains are distributed differently, such as a halo of small grains on eccentric orbits, has to be computed in a separate run with its own size distribution; images and SEDs of optically thin components add.
- **Effective-medium grains.** EMT is valid for inclusions much smaller than the wavelength. Large inclusions or aggregates need methods such as DDA or T-matrix codes.
- **Simulated observations.** Gaussian PSFs only, a sharp-edged mask, no noise and no instrumental polarisation. Use the noise-free images as input to an instrument simulator for a full prediction.
- **Optically thick regions.** DMS reports the line-of-sight optical depth (`image_tau`) and the largest radial optical depth in the mid-plane, and warns if the latter exceeds 0.1. Where these approach unity the results are no longer valid.
- **No extinction of the starlight.** In reality the light diffracted by grains in front of the star stays within $\approx 1/x$ rad of the star and comes with the extinction of the starlight. DMS computes neither, so for edge-on disks the scattered SED contains forward-scattered light that an observation would not separate from the star.

Part II
User guide

13 Installation

13.1 Prerequisites

- A Fortran compiler that supports Fortran 2008. The source files are free-form Fortran (`.f90`); version 2.0 also uses a few features introduced in Fortran 2003 and 2008, such as `error stop`, `open(newunit=...)`, `execute_command_line` and the `block` construct, and the `Makefile` compiles with `-std=f2008`. Tested with `gfortran` 10.5 and 13.3. The `Makefile` also has settings for Intel `ifx`, which has not been tested.
- GNU `make`. OpenMP is used if the compiler supports it.
- For the Python interface: Python 3.9 or later and NumPy 1.22 or later. With Python 3.12 or later, `numpy.f2py` also needs `meson` and `ninja`. `astropy` (FITS) and `matplotlib` (plots) are optional.
- For composite grains with `dms.emc`: the `emc` executable (Ossenkopf 1991), installed separately.
- For the test suite: `pytest`.

Install the compiler and `make` first:

System	Compiler and `make`
macOS	`xcode-select --install` (make, C compiler), then Homebrew (`https://brew.sh`) and `brew install gcc` (gfortran with OpenMP)
Debian, Ubuntu	`sudo apt install gfortran make python3 python3-pip python3-venv`
Fedora	`sudo dnf install gcc-gfortran make python3 python3-pip`
Windows	the Windows Subsystem for Linux (WSL) with Ubuntu, then as for Ubuntu

Any Python distribution works (Anaconda, Miniforge, python.org, the system Python); a separate environment keeps the versions consistent (`conda create -n dms python=3.12`, `conda activate dms`). The Python packages are listed in `requirements.txt`:

```
python3 -m pip install "numpy>=1.22" meson ninja matplotlib astropy pytest jupyterlab
# or, in the repository folder:  python3 -m pip install -r requirements.txt
```

Use the same Python for `make python`, for Jupyter and for scripts: the Python extension is compiled for one Python version (its file name contains it, e.g. `cpython-312`) and works only with that one.

13.2 Getting the code and building

```
git clone https://github.com/minjaegasparkim/dms.git
cd dms
make                        # optimised build with OpenMP   -> bin/dms
make python                 # Python extension (f2py)       -> python/dms/_dms*.so
make check                  # check the prerequisites and the built files
pip install -e python       # optional: "import dms" from anywhere
```

`make check` runs `tools/check_setup.py` (standard library only), which lists the compiler, `make`, Python, NumPy, `meson` and `ninja`, the optional packages, `bin/dms`, the Python extension for the Python that runs it, and whether `import dms` finds this repository or another installation. For every missing item it prints the command that installs it. Common problems:

Message	What to do
`gfortran: command not found`	install the compiler (table above)
`Cannot use distutils backend with Python>=3.12`, then the build stops	`python3 -m pip install meson ninja`, then `make python`
`ImportError: the compiled DMS extension is missing`	build it with the Python in use: `make python PYTHON=$(which python3)`
`module 'numpy' has no attribute ...`	NumPy is too old: `pip install --upgrade numpy`

Other targets of the `Makefile`:

Target	Effect
`make`	optimised program with OpenMP (`-O3 -march=native`)
`make debug`	run-time checks, floating-point traps, no optimisation
`make serial`	optimised, without OpenMP
`make python`	Python extension from the same library `build/libdms.a`
`make test`	build program and Python module, run the test suite
`make check`	check the prerequisites and the built files
`make clean`	remove build products
`make FC=ifx OMP=-qopenmp`	build with the Intel compiler

The option `-march=native` optimises for the machine that compiles the code. If the program is copied to another machine, build it there, or set `OPT="-O3"`.

13.3 Getting started with Jupyter

The folder `examples/` contains four notebooks. The first one checks the installation:

1. In a terminal, in the repository folder, start Jupyter: `jupyter lab`.
2. In the file browser on the left, open `examples/0_getting_started.ipynb`.
3. Run the cells one after the other with Shift + Enter.

The notebook runs `tools/check_setup.py` with the Python of the notebook, imports DMS from the repository, computes the SED of a small ring ($f_\mathrm{d} = 8.1\times10^{-5}$) and an image at 70 μm, and lists what to do if a step fails. It takes less than a minute. The other notebooks are `1_first_model.ipynb` (SED and images of a ring, changing a model; about ten seconds), `2_two_belts_observed.ipynb` (two belts of icy grains observed with a PSF and a coronagraph mask; about two minutes), `3_parameter_study.ipynb` (12 models in parallel with `dms.batch`; about half a minute on two cores) and `4_ice_loss.ipynb` (ice loss by sublimation and photosputtering; about half a minute); they are described in Sect. 16. On GitHub, the notebooks can be read with their results without running them.

13.4 Testing the installation

```
bin/dms examples/debris_ring.nml
python3 tools/quicklook.py results ring -o ring.png
```

The first command prints the progress and a summary, and writes the results to `results/`. The second makes a figure with the images, the SED and the optical-depth profile.

Test suite. With `pytest` installed (`pip install pytest`),

```
make test                          # build everything, run the tests
python3 -m pytest tests -v         # the same, test by test
```

runs the test suite in `tests/` (81 tests, about three minutes on two cores):

- **physical checks** with known answers: energy balance (integrated thermal SED = $f_\mathrm{d}L_\star$), thermal SED from images against the direct quadrature, belt mass fractions, a size distribution read from a file against the same power law, the Rayleigh limit and the Henyey–Greenstein polarisation law for a face-on ring, the rotation of the image with the position

angle, forward scattering on the near side, the radial extent of an eccentric belt, and the proportionality of all fluxes to the dust mass;

- **blow-out**: default threshold, release at pericentre and apocentre, user value, thresholds per belt, conflicting inputs;
- **temperature-dependent optical constants**: file names, duplicate temperatures, the input of version 2.0, several components, the modes `'nearest'` and `'linear'`, temperatures outside the sets, blow-out and sublimation with the set used at each distance;
- **DHS**: `dhs_xmax` against the full DHS, no effect on spheres, the optics cache;
- **photosputtering**: the switch, the ice-loss radius and the UV photon flux against analytic values, residue and vanishing grains, unbound grains, the reference times (collisions, P-R drag, age, shortest), a UV spectrum file, the interstellar field, the release orbits, and the eccentric belts with the ice decided per orbit (against the analytic orbit average, against the run without photosputtering when no ice is lost, the limit $e \to 0$, energy balance and images);
- **interface**: invalid input raises `dms.DMSError` without ending the session, repeated runs, namelist round trip, command-line program against Python, exit codes, the warning for coarse size grids;
- **example notebooks**: the code of notebooks 0, 1, 3 and 4 runs and gives the expected numbers (notebook 2, about two minutes, only with `DMS_TEST_SLOW=1`, which the continuous integration sets);
- **regression**: the eight examples, reduced in size, against the reference values in `tests/reference/examples.json` (relative tolerance 10^{-4}; differences between compilers and thread counts are at the level of 10^{-12}; tests that compare two runs of the same model use 10^{-10}, since the order of the OpenMP sums changes the far Wien tail by up to 2×10^{-12} on a busy machine).

After a change that alters results on purpose, regenerate the reference values with `python3 tests/regression.py --write` and explain the change in `CHANGELOG.md`.

Continuous integration. On GitHub, the workflow `.github/workflows/tests.yml` runs on every push and pull request (Ubuntu, Python 3.10 and 3.12): a debug build runs one example with run-time checks and floating-point traps, then the optimised program and the Python module are built and the test suite runs. The result is shown on the repository page under *Actions*.

14 Repository layout

Path	Content
`src/`	Fortran source: modules, routines and the main program `dms.f90`
`src/user_density.f90`	user-defined density distribution (edit and recompile)
`python/`	Python package `dms` and the f2py interface `dms_api.f90`
`examples/`	input files showing the main features, and five example notebooks (0–4)
`data/optical_constants/`	optical constants (wavelength [μm], n, k)
`data/stellar_models/`	model-atmosphere grids (built with `python -m dms.stellar`)
`docs/`	input reference, outputs, conventions, Python interface, this manual
`tests/`	test suite (pytest) and reference values of the examples
`tools/`	quick-look plots, optical-constant plots
`.github/workflows/`	continuous integration (build and tests on GitHub)
`results/`, `cache/`	created at run time (output files, saved optical properties)

All directories can be changed in the input file (`&run`: `data_dir`, `output_dir`, `cache_dir`). Relative paths are taken relative to the working directory, so the simplest way is to run DMS from the repository root.

15 Running dms

15.1 Command line

```
bin/dms model.nml              # input file (recommended)
bin/dms                        # interactive questions
bin/dms < answers.txt          # interactive questions answered from a file
bin/dms --help
export OMP_NUM_THREADS=8       # number of threads
```

15.2 What happens during a run

Figure 3 shows the sequence of a run. After every step DMS checks an error flag; on an error it prints a message and stops (command line) or returns to Python.

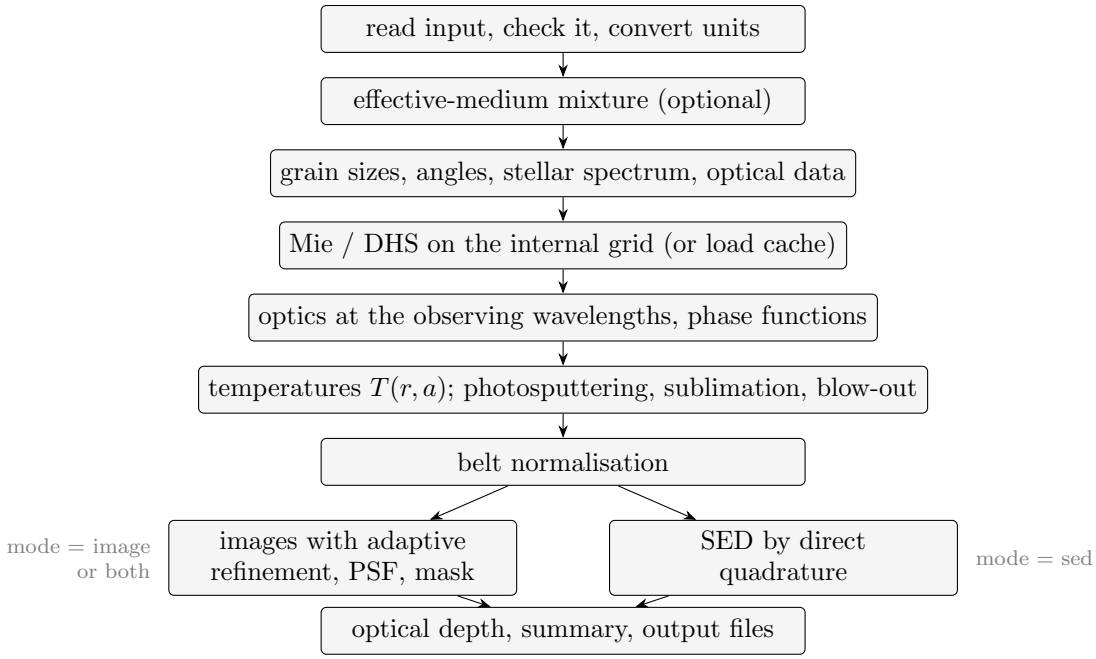


Figure 3: Sequence of a run (`src/dms_driver.f90`). Figure 4 gives the source files of every step.

Figure 4 shows the same sequence with the source files and the main routines of every step, the entry points (command line, Python, `dms.batch`), the modules used by all steps, and the Python modules used before and after a run. Its source is `docs/images/code_map.dot` (Graphviz).

15.3 A typical workflow

1. Start from the example closest to your problem and copy it.
2. Set the star, the disk and the dust. Choose the observing wavelengths inside the wavelength range of the optical data.
3. Run `mode = 'sed'` first: it is fast and checks f_{d} and the SED.

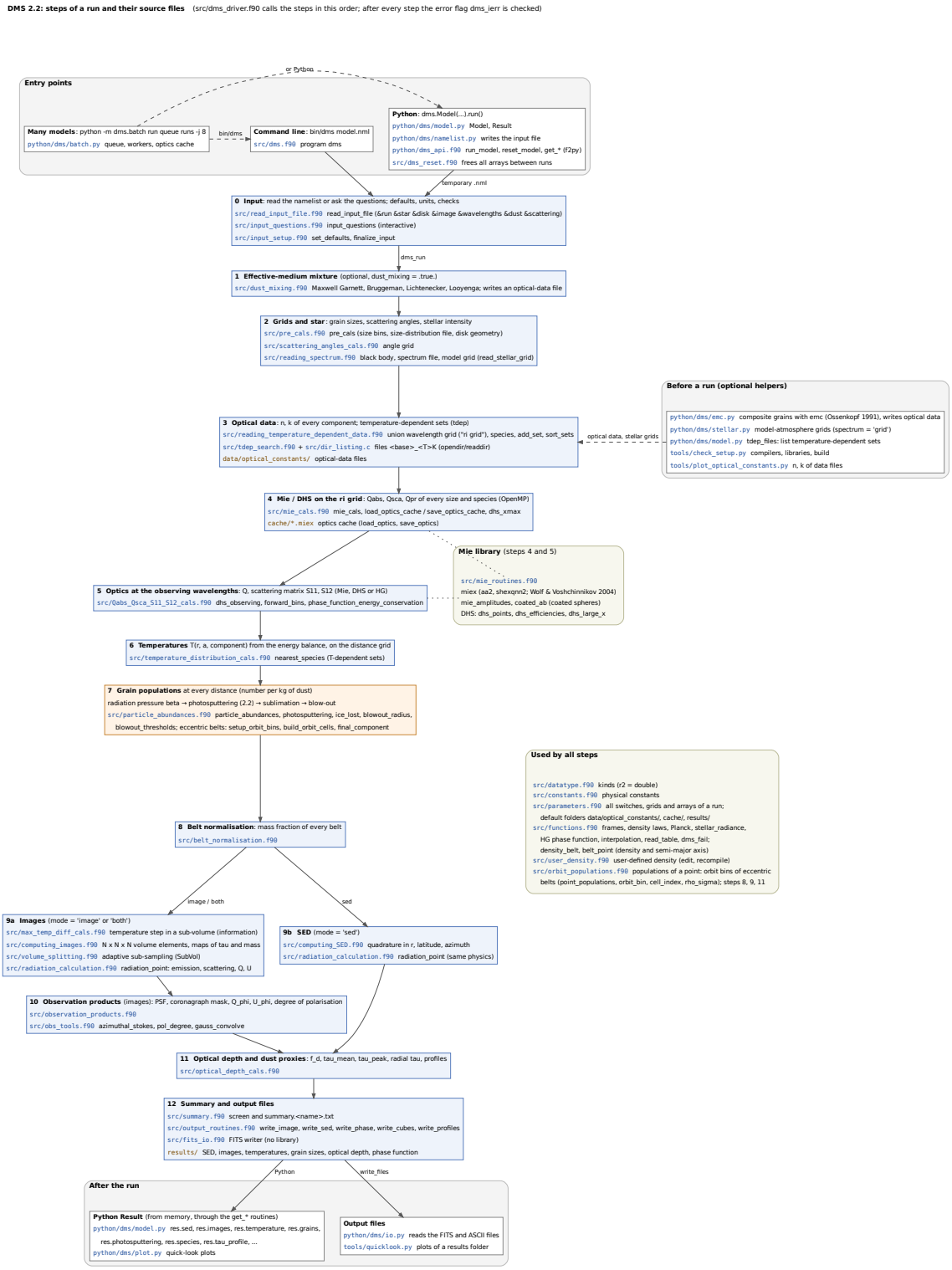


Figure 4: Steps of a run and their source files. The figure is a vector graphic: zoom in on screen, or open or print `docs/images/code_map.pdf` (also `.svg`, `.png`) at full size.

4. Switch on images with a small image (e.g. 101 pixels), check them with `tools/quicklook.py`, then increase the resolution.
5. Save the optical properties (`save_optics`) and reuse them (`load_optics`) while you vary the geometry or the star.

16 Examples: a cookbook

This chapter is a set of recipes, one for every common task. Each recipe names the files it uses, how long it runs on a laptop with two cores, and what it shows; then it gives the input that matters and the result. All files are in `examples/`, and every recipe runs as it is from the repository root after `make && make python`:

```
bin/dms examples/debris_ring.nml              # an input file: results in results/
python3 examples/recipes/ring_from_file.py    # a recipe script: prints numbers,
    writes a PNG
jupyter lab examples/4_ice_loss.ipynb         # a notebook
```

The recipe scripts in `examples/recipes/` are short (30–60 lines). They read the example input files with `dms.Model.from_file` or define the model in Python, run it, print the main numbers and write the figure shown here into the current directory. To start a new model, copy the recipe closest to your problem and change the star, the disk and the dust.

Table 5: Recipes. Times for two cores; the first run of a recipe also computes and caches the optical properties.

Sect.	Files	Task
16.1	`1_first_model.ipynb`	first model in Python: SED and images
16.2	`debris_ring.nml`, `recipes/ring_from_file.py`	a ring from an input file; images and SED
16.3	`eccentric_ring.nml`, `recipes/eccentric_ring.py`	eccentric ring: pericentre and apocentre glow
16.4	`two_belts.nml`, `2_two_belts_observed.ipynb`	two belts, ice sublimation, simulated observations
16.5	`polarisation_hg.nml`, `edge_on_disk.nml`, `recipes/observed_polarisation.py`	polarimetric imaging with PSF and coronagraph
16.6	`clump_user_density.nml`, `recipes/user_clump.py`	your own density distribution
16.7	—	composite and porous grains (EMT, `emc`)
16.8	`recipes/hollow_spheres.py`	non-spherical grains (DHS)
16.9	`recipes/forsterite_temperature.py`	temperature-dependent optical constants
16.10	`sublimation_ice.nml`, `recipes/sublimation.py`	sublimation of ice
16.11	`4_ice_loss.ipynb`, `photosputtering_ice.nml`	ice loss by photosputtering; eccentric belts
16.12	`3_parameter_study.ipynb`	a grid of models in parallel
16.13	—	fitting an observed SED

16.1 A first model in Python

Files: `examples/1_first_model.ipynb`
Run time: about 10 s
Shows: how to define a model in Python without an input file, run an SED and images, and plot them.

The notebook `examples/1_first_model.ipynb` defines a model in Python, without an input file, and computes an SED and two images in about ten seconds:

```
import dms

model = dms.Model(
    star={"teff": 5800, "radius": 1.0, "mass": 1.0, "distance": 20},    # K, R_sun,
    M_sun, pc
    disk={"r_in": 50, "r_out": 100, "dust_mass": 1e-8,                   # au, M_sun
          "inclination": 60, "position_angle": 30},                      # degrees
    dust={"n_components": 1, "optical_data": "astrosilicate_draine03_862",
          "bulk_density": 3.5, "a_min": 1, "a_max": 1000, "q": 3.5})     # g/cm^3,
    micron

sed = model.copy().update(run={"mode": "sed"},
          wavelengths={"lambda_min": 1, "lambda_max": 2000, "n_lambda": 60}).run()
img = model.copy().update(run={"mode": "image"}, wavelengths={"lambda": [1.6, 870]},
          image={"n_pixels": 151}).run()
print(sed.f_d)                       # 8.1e-05
```

Parameters that are not given keep their defaults (Sect. 17). The notebook then plots the SED and the images at 1.6 and 870 μm with `dms.plot` (Fig. 5) and shows how the image changes when the ring is made eccentric.

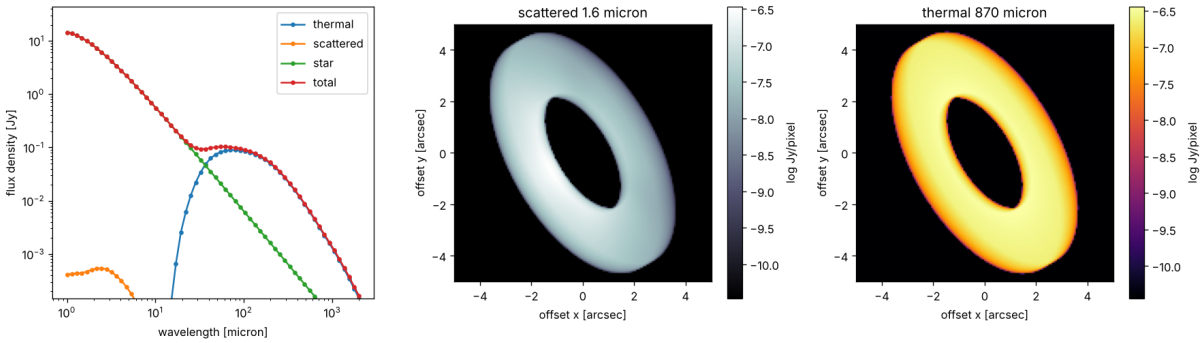


Figure 5: Output of `examples/1_first_model.ipynb`: SED, scattered light at 1.6 μm and thermal emission at 870 μm (logarithmic scale, Jy per pixel) of a ring between 50 and 100 au around a Sun-like star at 20 pc.

16.2 A ring from an input file: images and SED

Files: `examples/debris_ring.nml`, `examples/recipes/ring_from_file.py`
Run time: about 10 s
Shows: the input-file route; images at four wavelengths, the SED from the images (`mode = 'both'`), a full SED by changing two inputs, and the optics cache.

```
&run    mode = 'both',  emission = 'both',  output_format = 'fits',  output_name = 'ring'
        load_optics = .true.,  save_optics = .true.,  optics_cache_name = 'astrosil_1-1000_n100
    ' /
&star   spectrum = 'blackbody',  teff = 5800.,  radius = 1.0,  mass = 1.0,  distance = 20.0 /
&disk   density_law = 'powerlaw',  alpha = 1.0,  r_in = 50.,  r_out = 100.
        vertical = 'wedge',  opening_angle = 5.0,  dust_mass = 1.0e-8
        inclination = 60.,  position_angle = 30. /
&image  n_pixels = 201,  max_delta_t = 2.0,  n_split_max = 3 /
&wavelengths  lambda = 1.6, 24., 70., 870. /
&dust   n_components = 1,  optical_data = 'astrosilicate_draine03_862',  bulk_density = 3.5
        size_distribution = 'powerlaw',  a_min = 1.0,  a_max = 1000.,  q = 3.5
        n_sizes = 100       ! >= ~30 bins per decade for smooth Mie polarisation
        blowout = .true. /
&scattering  n_angles = 181,  phase_function = 'mie' /
```

From the command line, `bin/dms examples/debris_ring.nml` writes the results to `results/` with the suffix `.ring`: FITS cubes `image_*.ring.fits` with four wavelengths, `SED.ring.dat`, `optical_depth.ring.dat`, the temperatures, the grain sizes and `summary.ring.txt`. The Mie results are saved in `cache/astrosil_1-1000_n100.miex` and reused in the next run. The recipe script does the same from Python and adds a full SED of the same model, changing only the run mode and the wavelengths:

```
m   = dms.Model.from_file("examples/debris_ring.nml")
res = m.run()                                          # images and the SED from them
sed = m.copy().update(run={"mode": "sed"},
          wavelengths={"lambda_min": 0.5, "lambda_max": 2000, "n_lambda": 80}).run()
img = res.image("thermal")[3]                          # 870 micron, Jy per pixel
```

It prints $f_\mathrm{d} = 8.1 \times 10^{-5}$ and the flux densities, e.g. 89 mJy at 70 μm and 1.8 mJy at 870 μm (Fig. 6).

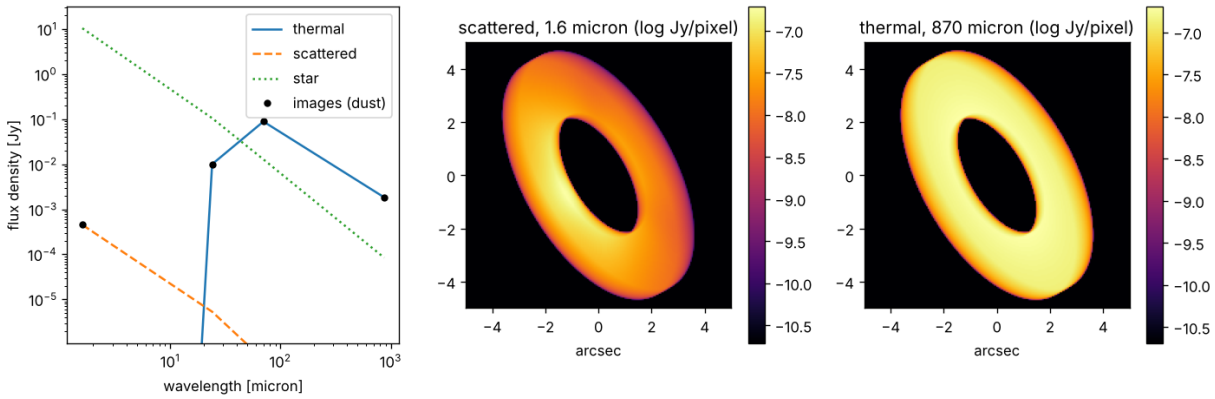


Figure 6: Output of `examples/recipes/ring_from_file.py`: SED (lines; dots: the four image wavelengths) and the images in scattered light at 1.6 μm and in thermal emission at 870 μm (logarithmic, Jy per pixel) of a ring at 50–100 au around a Sun-like star at 20 pc, $i = 60°$.

16.3 An eccentric ring: pericentre and apocentre glow

Files: `examples/eccentric_ring.nml`, `examples/recipes/eccentric_ring.py`

Run time: about 5 s

Shows: an eccentric ring ($e = 0.3$, semi-major axes 130–145 au, face-on) around an A star; the azimuthal profile of the flux.

```
&disk   density_law = 'powerlaw',  alpha = 0.
        r_in = 130.,  r_out = 145.              ! semi-major axes [au]
        eccentricity = 0.3,  omega = 0.         ! pericentre along +Y
        opening_angle = 2.,  dust_mass = 5.0e-8,  inclination = 0. /
&wavelengths  lambda = 24., 870. /
```

The recipe sums the image in 15° bins of azimuth around the star. At 24 μm the pericentre side is 2.4 times brighter than the apocentre side, because its dust is closer to the star and warmer (pericentre glow). At 870 μm the ratio is 0.43: the dust moves slowly near apocentre and piles up there, and the emission in the Rayleigh–Jeans regime depends only weakly on the temperature (apocentre glow; Fig. 7). Section 2 gives the geometry.

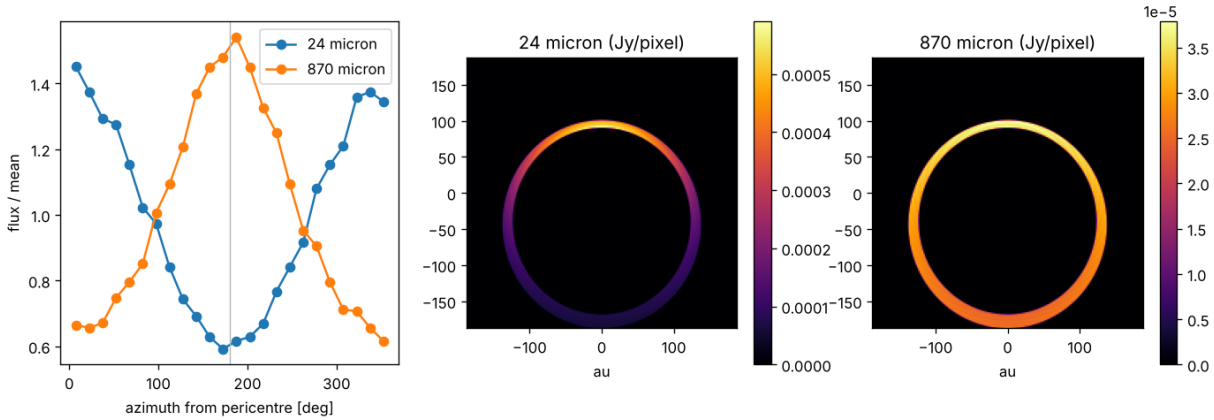


Figure 7: Output of `examples/recipes/eccentric_ring.py`: flux in 15° azimuth bins (relative to the mean) against the azimuth from pericentre, and the images at 24 and 870 μm (pericentre at the top).

16.4 Two belts, sublimation and simulated observations

Files: `examples/two_belts.nml`, `examples/2_two_belts_observed.ipynb`
Run time: a few seconds (input file); about two minutes (notebook)
Shows: several belts with their own density laws and eccentricities, mass fractions, a Gaussian vertical profile; in the notebook also icy grains that sublimate, blow-out for release at pericentre and observations with PSFs and a coronagraph mask.

```
&disk  n_belts = 2
       !                      belt 1 (warm)   belt 2 (cold)
       density_law          = 'powerlaw',     'gaussian'
       alpha                = 0.0,            0.0
       mu                   = 0.0,            140.
       sigma                = 0.0,            8.
       r_in                 = 5.,             110.
       r_out                = 12.,            170.
       eccentricity         = 0.0,            0.12
       omega                = 0.0,            40.
       belt_mass_fraction = 0.05,             0.95
       vertical = 'gaussian',  h_over_r = 0.03,  dust_mass = 2.0e-7
       inclination = 66.,  position_angle = 156. /
```

The warm belt holds 5 per cent of the dust mass. The cold belt is eccentric, so its pericentre side is warmer and brighter in thermal emission.

The notebook `examples/2_two_belts_observed.ipynb` (about two minutes) uses more of the code: a warm inner belt (5–15 au) and a cold eccentric outer belt (Gaussian, peak at 140 au, $e = 0.12$) around an A star at 7.7 pc; grains of silicate and water ice that turn into porous silicate above 110 K; blow-out for release at pericentre; a polarised-light image with a PSF and a coronagraph mask; thermal images with a PSF. The dust and the three runs are

```
dust={"n_components": 2,
      "optical_data": ["kim2019/ice_astrosilicate_MG/Sil50cice50",   # silicate + ice
                       "kim2019/porous_astrosilicate/as50vac50"],    # residue
      "bulk_density": [2.25, 1.75], "abundance": [100, 0],
      "sublimation": True, "t_sublimation": [110, -1], "sublimation_residue": [2, 0],
      "a_min": 1, "a_max": 1000, "q": 3.5, "blowout_release": "pericentre"}

sed = system.copy().update(run={"mode": "sed"},
          wavelengths={"lambda_min": 1, "lambda_max": 2000, "n_lambda": 80}).run()
pol = system.copy().update(run={"mode": "image", "emission": "polarisation"},
          wavelengths={"lambda": [1.6]},
          image={"n_pixels": 201, "psf": "gaussian", "psf_fwhm": 0.2, "mask_radius":
    2.5}).run()
th  = system.copy().update(run={"mode": "image", "emission": "thermal"},
          wavelengths={"lambda": [24, 870]},
          image={"n_pixels": 201, "psf": "gaussian", "psf_fwhm": [0.8, 0.4]}).run()
```

The blow-out threshold is $\beta \geq 0.44$ in the outer belt ($(1 - e)/2$) and 0.5 in the inner one; icy grains smaller than about 5 μm are removed. The ice of the remaining grains sublimates inside 21–27 au, so the inner belt consists of porous silicate. Figure 8 shows the results; the notebook also prints $f_\mathrm{d} = 1.0 \times 10^{-4}$, the largest radial optical depth (2.8×10^{-3}) and the flux densities.

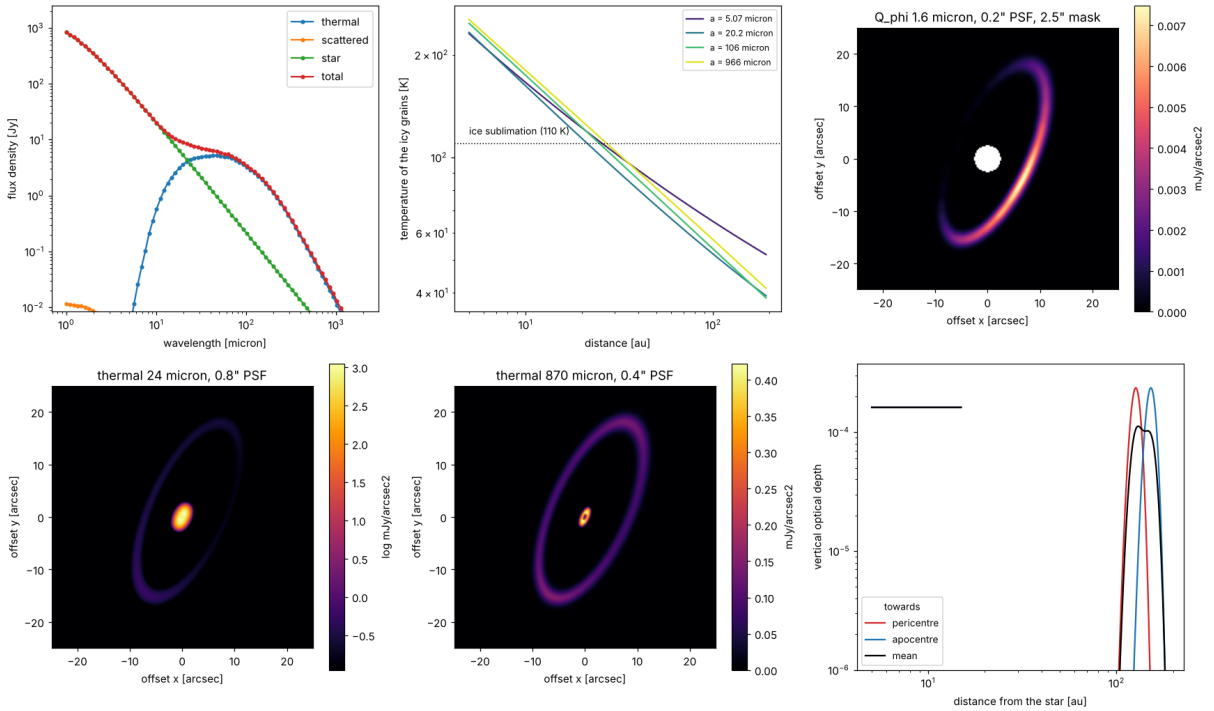


Figure 8: Output of `examples/2_two_belts_observed.ipynb`. Top: SED; temperature of the icy grains for four sizes (the smallest one left after blow-out, 20, 100 and 1000 μm) with the sublimation temperature; Q_ϕ at 1.6 μm convolved with a 0.2″ PSF, with a coronagraph mask of 2.5″ radius. Bottom: thermal emission at 24 μm (logarithmic scale) and 870 μm convolved with PSFs of 0.8″ and 0.4″; vertical optical depth towards pericentre and apocentre of the outer belt and its azimuthal mean.

16.5 Polarimetric imaging with a PSF and a coronagraph

Files: `examples/polarisation_hg.nml`, `examples/edge_on_disk.nml`, `examples/recipes/observed_polarisation.py`

Run time: about 30 s

Shows: scattered and polarised light with a two-component Henyey–Greenstein phase function, a diffraction-limited PSF, a coronagraph mask and Q_ϕ in mJy arcsec^{-2}.

```
&run    mode = 'image',  emission = 'polarisation',  output_name = 'pol_hg' /
&disk   density_law = 'broken',  alpha = 10.,  alpha_out = -4.,  r_c = 70.
        r_in = 40.,  r_out = 140.,  vertical = 'gaussian',  h_over_r = 0.04
        dust_mass = 1.0e-8,  inclination = 75.,  position_angle = 20. /
&image  n_pixels = 201,  psf = 'diffraction',  telescope_diameter = 8.2
        mask_radius = 0.1,  image_unit = 'mJy/arcsec2' /
&wavelengths  lambda = 1.25, 1.65 /
&scattering  phase_function = 'hg2',  g1 = 0.8,  g2 = -0.2,  w = 0.75,  p_max = 0.5 /
```

This mimics a near-infrared polarimetric observation with an 8-m telescope: diffraction-limited PSF, a 0.1″ coronagraph and Q_ϕ images. The recipe plots the observed Q_ϕ at 1.65 μm (peak 0.04 mJy arcsec^{-2}) and the model degree of polarisation (Fig. 9). `edge_on_disk.nml` is the same kind of observation for a disk seen at $i = 88°$ with a smooth broken power law and a Gaussian vertical profile. For Mie polarisation instead of the Henyey–Greenstein law, remove the `&scattering` group (`phase_function = 'mie'` is the default); Sect. 16.8 shows how the grain shape changes it.

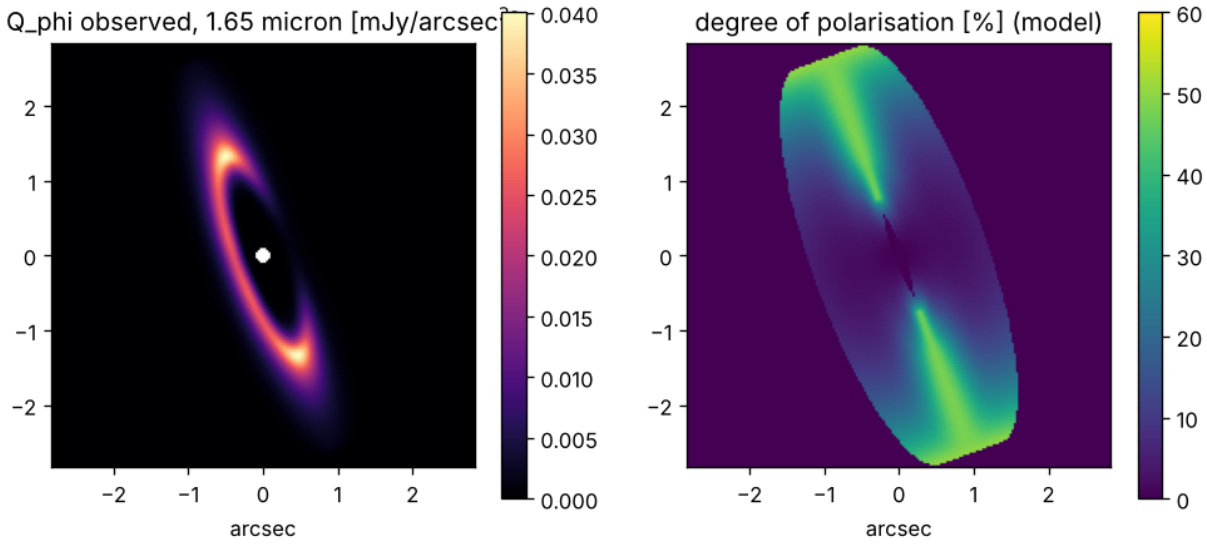


Figure 9: Output of `examples/recipes/observed_polarisation.py`: Q_ϕ at 1.65 μm convolved with the PSF of an 8.2-m telescope and masked inside 0.1″, and the degree of polarisation of the model.

16.6 Your own density distribution

Files: `examples/clump_user_density.nml`, `src/user_density.f90`, `examples/recipes/user_clump.py`

Run time: about 5 s

Shows: a density law of your own (here a clump in azimuth) and an image convolved with a beam.

```
&disk   density_law = 'user'
        user_par = 1.0, 3.0, 45.0, 15.0    ! alpha, clump amplitude, azimuth [deg], width [deg]
        r_in = 40.,  r_out = 60.,  opening_angle = 3.,  dust_mass = 1.0e-8
        inclination = 30.,  position_angle = 0. /
&image  n_pixels = 201,  psf = 'gaussian',  psf_fwhm = 0.3,  image_unit = 'mJy/arcsec2' /
&wavelengths  lambda = 870., 1300. /
```

With `density_law = 'user'` DMS calls `user_density_fn` in `src/user_density.f90`; the default is a power law multiplied by a Gaussian clump in azimuth, with the four parameters `user_par`. Write your own function there and run `make` (and `make python`). The recipe gives 9.6 mJy at 870 μm and the images convolved with a 0.3″ beam (Fig. 10).

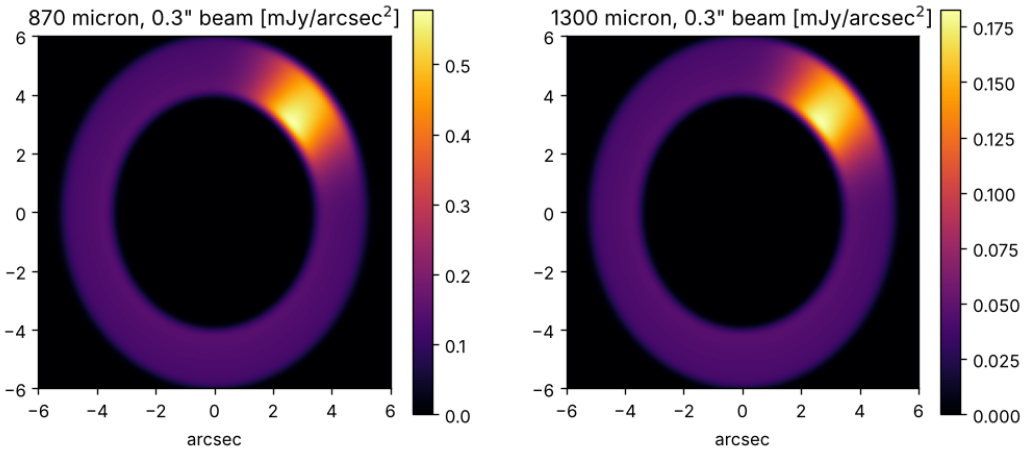


Figure 10: Output of `examples/recipes/user_clump.py`: a ring with a clump at an azimuth of 45°, at 870 and 1300 μm, convolved with a 0.3″ beam.

16.7 Composite and porous grains

Shows: two ways to make an optical-data file for a mixture.

DMS mixes two materials itself with effective-medium theory (Sect. 4.2): give the matrix, the inclusion and the volume fraction in `&dust` and use the file it writes as a component (here 50 % amorphous ice in astronomical silicate, Bruggeman rule):

```
&dust   emt = .true.,  emt_rule = 'bruggeman'
        emt_matrix = 'astrosilicate_draine03_862'
        emt_inclusion = 'kim2019/pure_ices/composite_aice.txt',  emt_volume_fraction = 0.5
        emt_output = 'as_aice_bg.txt'          ! written to data_dir
        n_components = 1,  optical_data = 'as_aice_bg.txt',  bulk_density = 2.25 /
```

For more components, inclusion shapes, coated grains and porosity, `dms.emc` runs the `emc` code (Sect. 20.2). Mixtures used in published work are also in `data/optical_constants/kim2019/`, e.g. `ice_astrosilicate_MG/Sil50cice50` (Maxwell Garnett, 50 % crystalline ice) and the porous residue `porous_astrosilicate/as50vac50`, used in Sects. 16.4, 16.10 and 16.11.

16.8 Non-spherical grains: hollow spheres

Files: `examples/recipes/hollow_spheres.py`
Run time: about 15 s
Shows: compact Mie spheres against a distribution of hollow spheres (DHS) in the SED and in the degree of polarisation against scattering angle.

```
for shape in ("sphere", "dhs"):
    m = base.copy().update(dust={"grain_shape": shape, "dhs_fmax": 0.8},
                           run={"optics_cache_name": f"recipe_{shape}"})
    res[shape] = m.run()
    angle, f_sca, f_pol = res[shape].phase_curve(0)    # whole disk, 1.6 micron
```

The model is an edge-on ring of astronomical silicate (10–30 au, 0.3–100 μm) around a Sun-like star. The hollow spheres absorb more per unit mass ($f_\mathrm{d} = 5.6 \times 10^{-4}$ against 5.1×10^{-4}) and polarise very differently: the spheres reach −40 % (parallel to the scattering plane) at 150°, while the hollow spheres give +40 % near 90° (Fig. 11), closer to what is observed for debris disks. Sect. 4.3 gives the method and `dhs_xmax` the treatment of large grains.

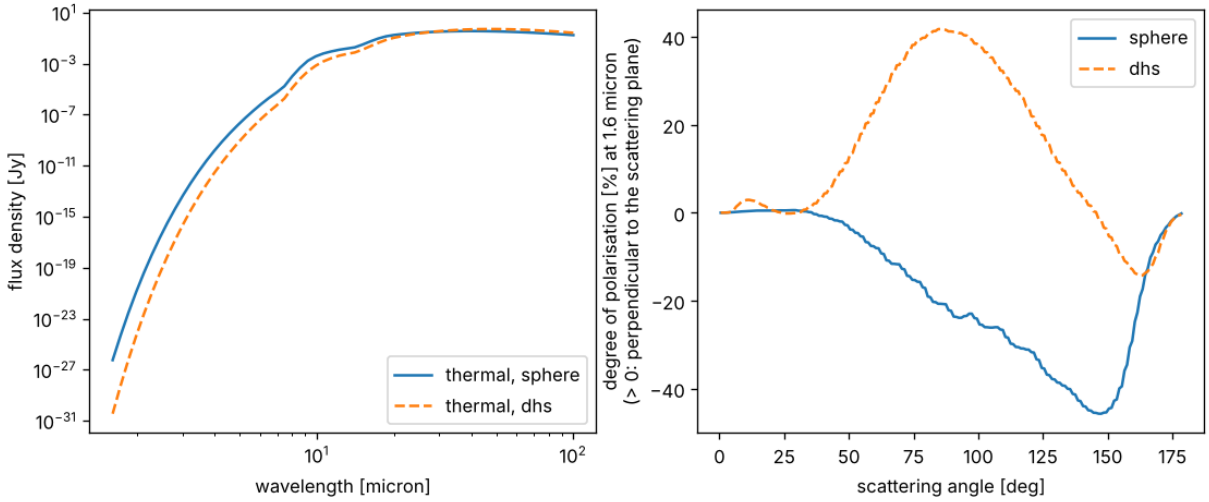


Figure 11: Output of `examples/recipes/hollow_spheres.py`: thermal SED and degree of polarisation of the whole disk against scattering angle at 1.6 μm (positive: perpendicular to the scattering plane) for compact spheres and for hollow spheres with $f_{\max} = 0.8$.

16.9 Temperature-dependent optical constants

Files: `examples/recipes/forsterite_temperature.py`, `data/optical_constants/suto2006/`
Run time: about 35 s
Shows: components whose optical constants depend on the grain temperature (`tdep`), in the modes `'nearest'` and `'linear'`.

The three crystal axes of forsterite are three components, each with five sets measured at 50, 100, 150, 200 and 295 K (Suto et al. 2006). With `tdep = .true.` DMS finds the files `fo_suto2006_A_<T>K.dat` from the base name and gives every grain the set nearest to its own temperature, or n and k interpolated in temperature (Sect. 6.2):

```
dust = {"n_components": 3, "bulk_density": [3.3] * 3, "abundance": [1.0] * 3,
        "optical_data": ["suto2006/fo_suto2006_A", "suto2006/fo_suto2006_B",
                         "suto2006/fo_suto2006_C"],
        "tdep": [True] * 3, "tdep_mode": "linear", "tdep_dt": 10.0}
```

The grains of a belt at 30–60 au around an A star are at 56–89 K. With the sets at their temperature the 69 μm band is much narrower and stronger and peaks at 68.8 μm, against 69.5 μm with the 295 K set alone (Fig. 12). `res.species` lists the optical species used: 15 data sets in `'nearest'`, 78 species in `'linear'` with steps of at most 10 K.

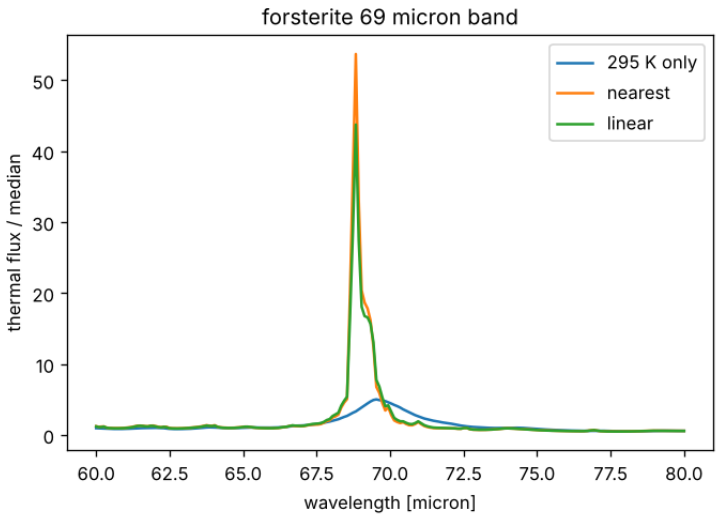


Figure 12: Output of `examples/recipes/forsterite_temperature.py`: the 69 μm forsterite band of a belt at 30–60 au around an A star with the 295 K data only, and with the data at the grain temperature (`'nearest'`, `'linear'`).

16.10 Sublimation of ice

Files: `examples/sublimation_ice.nml`, `examples/recipes/sublimation.py`
Run time: about 15 s
Shows: a component that turns into another one where it is warmer than its sublimation temperature, and the sublimation radius of every grain size.

```
&run    mode = 'sed',  output_name = 'sublimation' /
&star   teff = 8590.,  radius = 1.84,  mass = 1.92,  distance = 7.7 /
&disk   density_law = 'powerlaw',  alpha = 1.0,  r_in = 3.,  r_out = 150.
        opening_angle = 5.,  dust_mass = 1.0e-7 /
&wavelengths  lambda_min = 1.0,  lambda_max = 1000.,  n_lambda = 120 /
&dust   n_components = 2
        optical_data = 'kim2019/ice_astrosilicate_MG/Sil50cice50',
                       'kim2019/porous_astrosilicate/as50vac50'
        bulk_density = 2.25,  1.75
        abundance    = 100.,  0.          ! the porous silicate only forms by sublimation
        a_min = 1.0,  a_max = 1000.,  q = 3.5,  n_sizes = 100
        sublimation = .true.
        t_sublimation       = 110.,  -1.
        sublimation_residue = 2,     0 /
```

Inside the distance where the grains reach 110 K, the ice–silicate grains become porous silicate grains. The recipe reads the temperatures (`res.temperature["T"][distance, component, size]`) and finds the sublimation radius of every size: 52 au for 1 μm grains, which are the hottest, a minimum of 21 au near 15 μm and 26 au for large grains (Fig. 13).

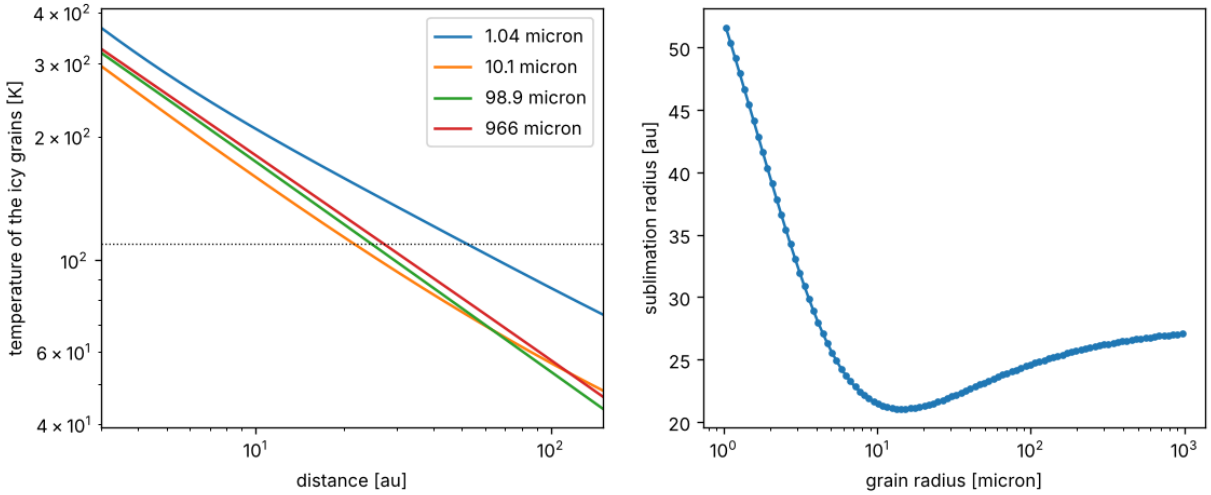


Figure 13: Output of `examples/recipes/sublimation.py`: temperature of the icy grains of four sizes against distance (dotted: 110 K) and the sublimation radius against grain size, around an A star.

16.11 Ice loss by photosputtering

Files: `examples/4_ice_loss.ipynb`, `examples/photosputtering_ice.nml`
Run time: about 45 s
Shows: ice loss by UV photons against collisions, P-R drag and age; release orbits; an eccentric belt with the ice decided per orbit.

Grains of silicate with 50 % crystalline water ice by volume (Kim et al. 2019; ice mass fraction 0.22) in a disk from 3 to 200 au around a solar-type star (black body, 10 pc, dust mass $10^{-7}\,M_\odot$) lose their ice by sublimation at 110 K and by photosputtering (Sect. 6.4); the ice-free remainder becomes porous silicate. Four models are compared: sublimation only; photosputtering against the shortest of the collisional lifetime (with the vertical optical depth `tau_peak` of the first model) and the P-R drag time, with the local UV and with the UV of the release orbits; and photosputtering against P-R drag alone, the limit of a faint disk.

```
PS = {"photosputtering": True, "ps_yield": [1e-3, 0.0],
      "ps_ice_fraction": [0.222, 1.0], "ps_residue": [2, 0]}
subl = disk.run()
ps = disk.copy().update(dust={**PS, "ps_time": "shortest", "ps_tau": subl.tau_peak,
                              "ps_orbit": "release"}).run()
ps.photosputtering["r_ice_loss"][:, 0]      # ice lost out to this distance [au],
    every size
```

The smallest bound grains lose their ice everywhere, because their sputtering time is shorter than every reference time; larger grains keep it outside a few to a few tens of au, where collisions destroy them first (Fig. 15). Against P-R drag alone, every size loses its ice everywhere. The scattered light, which comes mostly from the small grains, changes by 10–40 % below 2 μm, and the 3 μm ice band disappears.

The second part of the notebook takes an eccentric belt (a = 30–60 au, e = 0.3, ice lost if sputtered within 10^5 yr) and compares `ps_orbit = 'local'` with `'release'`, where the ice is decided per orbit (32 orbit bins). In Fig. 14 the ratio of the scattered light at 3.1 and 2.0 μm follows the distance from the star with the local decision and the orbits of the belt with the decision per orbit.

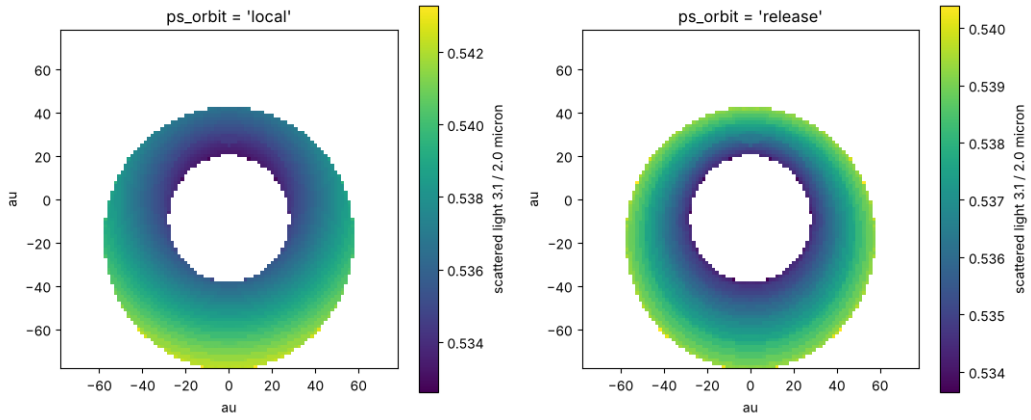


Figure 14: Example 4, eccentric belt seen face-on (pericentre at the top, star at the origin): ratio of the scattered light at 3.1 and 2.0 μm with the ice decided at every point (`ps_orbit = 'local'`) and per orbit (`'release'`).

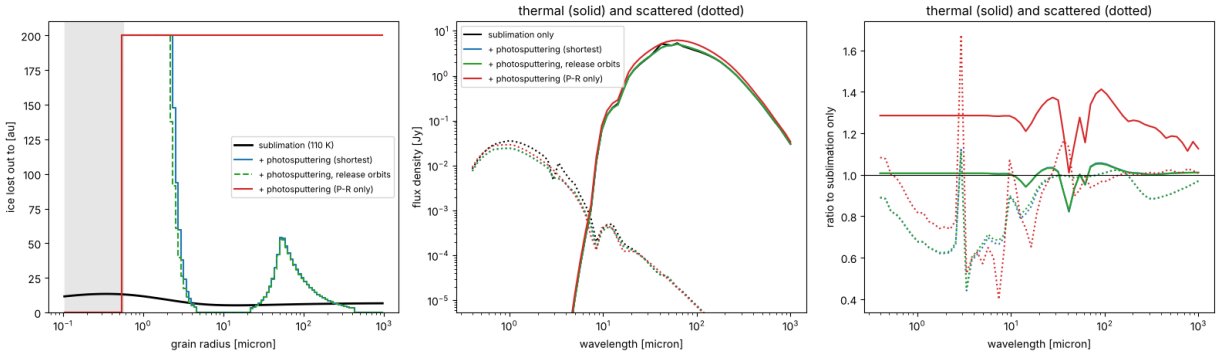


Figure 15: Output of `examples/4_ice_loss.ipynb`: the largest distance at which grains lose their ice by photosputtering against grain size (black: sublimation radius; shaded: grains below the blow-out size), the SEDs of the four models (solid: thermal, dotted: scattered), and their ratio to the model with sublimation only.

16.12 A grid of models in parallel

Files: `examples/3_parameter_study.ipynb`
Run time: about 30 s
Shows: `dms.batch`: a grid of 12 models from one base model, linked parameters, parallel workers, collecting the results.

The notebook `examples/3_parameter_study.ipynb` computes 12 models of a nearly edge-on ring ($i = 88°$, 2.5–3.8 μm): three grain compositions (astronomical silicate and Maxwell–Garnett mixtures with 20 and 50 % crystalline water ice), a G and an A star, and $q = 3.5$ and 3.0. The radius and mass of each star change together with its temperature through `linked`:

```
jobs = batch.grid(base,
    {"dust.optical_data": ["kim2019/astrosilicate/astrosilicate_draine2003.txt",
                           "kim2019/ice_astrosilicate_MG/Sil80cice20",
                           "kim2019/ice_astrosilicate_MG/Sil50cice50"],
     "star.teff": [5800, 8590],
     "dust.q": [3.5, 3.0]},
    linked={"dust.optical_data": {"dust.bulk_density": [3.5, 3.0, 2.25]},
            "star.teff": {"star.radius": [1.0, 1.84], "star.mass": [1.0, 1.92]}})
batch.write_queue(jobs, "parameter_study/queue")
```

```
batch.run("parameter_study/queue", "parameter_study/runs", workers=8, omp_threads=1)
rows = batch.collect("parameter_study/runs")
batch.dop_at(rows[0], 90)        # degree of polarisation at 90 deg, every wavelength
```

The notebook takes about half a minute on two cores; run again, it computes only the models that are missing. It prints the smallest grain left after blow-out (0.5–0.6 μm for the G star, 3.3–4.5 μm for the A star), $f_{\rm d}$ and the degree of polarisation at 2.5 and 3.1 μm, and plots Fig. 16. For the G star and $q = 3.5$, the degree of polarisation at 90° and 3.1 μm is 9 % for silicate, 46 % with 20 % ice and 68 % with 50 % ice.

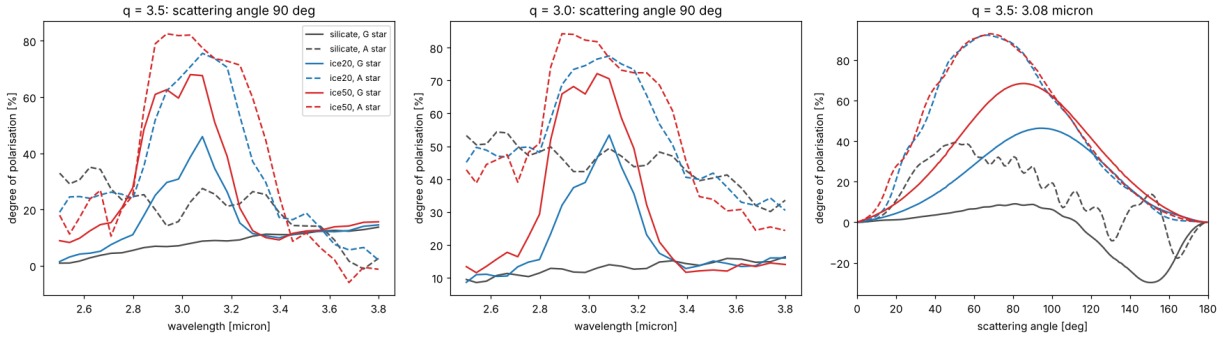


Figure 16: Output of `examples/3_parameter_study.ipynb`: degree of polarisation at a scattering angle of 90° against wavelength for $q = 3.5$ and 3.0, and against the scattering angle at 3.1 μm for $q = 3.5$ (solid: G star, dashed: A star). With the A star the size distribution starts at the blow-out size, and its sharp lower edge leaves Mie oscillations in the curves.

16.13 Fitting an observed SED

Shows: a small grid fitted to an SED with `multiprocessing`.

Section 19.3 gives a complete script: it computes the optical properties once, then evaluates χ^2 for a grid of inner radii and density exponents in parallel processes. For larger grids, use `dms.batch` (Sect. 16.12); for MCMC, call the same model function from `emcee`.

17 Input reference

An input file is a Fortran namelist with seven groups: `&run`, `&star`, `&disk`, `&image`, `&wavelengths`, `&dust` and `&scattering`. All groups are optional and may come in any order; a parameter that is not given keeps its default. Comments start with `!`; strings are written in quotes, logicals as `.true.`/`.false.`, and lists are separated by commas.

Units: lengths in au, angles in degrees, masses in $M_\odot$, stellar radius in $R_\odot$, distance in pc, grain radii and wavelengths in μm, temperatures in K.

17.1 `&run`: what to compute and where to write it

Name	Default	Meaning
`mode`	`'image'`	`'image'`; `'sed'` (direct integration, no images); `'both'` (images plus an SED from the summed images)

Name	Default	Meaning
`emission`	`'both'`	`'thermal'`, `'scattering'`, `'both'`, `'polarisation'`
`output_format`	`'fits'`	`'ascii'`, `'fits'`, `'both'` (images)
`output_name`	`''`	appended to all output file names
`name_with_parameters`	`.false.`	also append main model parameters to file names
`data_dir`	`'data/optical_constants/'`	directory of the optical data
`output_dir`	`'results/'`	output directory (created if missing)
`cache_dir`	`'cache/'`	directory of saved optical properties
`load_optics`	`.false.`	reuse saved optical properties if radii, wavelengths, angles, grain shape and n, k are identical
`save_optics`	`.false.`	save the optical properties
`optics_cache_name`	`'dms_optics'`	cache file name (in `dms.batch`, `'auto'` derives it from the dust, wavelength and angle parameters)
`save_temperature`	`.true.`	write the temperature distribution
`save_grain_sizes`	`.true.`	write grain numbers, cross sections and β
`screen_output`	`.true.`	print progress
`write_files`	`.true.`	write output files (Python sets `.false.` by default)

17.2 `&star`

Name	Default	Meaning
`spectrum`	`'blackbody'`	`'blackbody'`, `'file'`, `'grid'`
`teff`	5800	effective temperature [K] (black body or grid)
`logg`	4.5	$\log g$ [cgs] (grid)
`radius`	1.0	stellar radius [$R_\odot$]
`mass`	1.0	stellar mass [$M_\odot$] (blow-out size)
`distance`	10.0	distance [pc]
`spectrum_file`		two columns: wavelength [μm], flux density [Jy] at `distance`
`grid_dir`		directory with `grid_index.txt` (Sect. 20.3)

17.3 `&disk`

Parameters marked *per belt* are lists with one value per belt.

Name	Default	Meaning
`n_belts`	1	number of belts (at most 10)
`density_law`	`'powerlaw'`	per belt: `'powerlaw'`, `'ertel'`, `'gaussian'`, `'broken'`, `'user'`
`r_in`, `r_out`	required	per belt: inner and outer semi-major axis [au]
`eccentricity`	0	per belt: $0 \le e < 1$
`omega`	0	per belt: argument of pericentre [deg], from the line of nodes ($+Y$) towards the far side ($+X$)
`belt_mass_fraction`	1	per belt: relative dust mass (normalised)
`alpha`	1.0	per belt: exponent of the radial law

Name	Default	Meaning
`alpha_out`	-3.0	per belt: outer exponent (`'broken'`)
`r_c`		per belt: characteristic radius [au] (`'broken'`)
`r_start`, `r_peak`		per belt [au] (`'ertel'`)
`mu`, `sigma`		per belt: centre and width [au] (`'gaussian'`)
`user_par`	0	ten parameters for `user_density_fn`
`user_density_symmetric`	`.false.`	user density symmetric under $Y \to -Y$ (allows mirror symmetry)
`vertical`	`'wedge'`	`'wedge'` or `'gaussian'`
`opening_angle`	5.0	half opening angle of the wedge [deg]
`h_over_r`	0.05	aspect ratio of the Gaussian profile
`gauss_cut`	4.0	cut of the Gaussian profile in scale heights
`dust_mass`	10^{-8}	surviving dust mass [$M_\odot$] between $a_{\min}$ and $a_{\max}$
`inclination`	0	0 = face-on, 90 = edge-on [deg]
`position_angle`	0	of the projected major axis, counter-clockwise from $+y$ [deg]

17.4 `&image`

Name	Default	Meaning
`n_pixels`	101	pixels per axis (an even number is increased by one)
`max_delta_t`	2.0	largest temperature difference inside an element [K]
`n_split_max`	3	maximum refinement level $N_{\max}$ (0–30)
`add_star`	`.false.`	add the stellar flux to the central pixel of the total image
`psf`	`'none'`	`'none'`, `'gaussian'`, `'diffraction'`
`psf_fwhm`		FWHM [arcsec]: one value, or one per wavelength
`telescope_diameter`		D [m] for `'diffraction'`
`mask_radius`	0	radius of an inner mask [arcsec]
`image_unit`	`'Jy/pixel'`	`'Jy/pixel'`, `'mJy/arcsec2'`, `'Jy/arcsec2'`
`save_3d_cubes`	`.false.`	write the flux of every volume element and its scattering angle as 3-d cubes; needs $16\, n_\lambda N^3$ bytes of memory (at most 8 GB)

17.5 `&wavelengths`

Give either a list, `lambda = 1.6, 24., 70.`, or a logarithmic range with `lambda_min`, `lambda_max` and `n_lambda`. All wavelengths must lie inside the range shared by the optical-data files.

17.6 `&dust`

Name	Default	Meaning
`n_components`	required	number of materials (at most 50)
`optical_data`		one file per component, relative to `data_dir`; the order does not matter (union wavelength grid)

Name	Default	Meaning
bulk_density		one value per component [g cm^{-3}]
abundance	1	relative number (= volume) abundance per component, normalised; 0 for a component that only forms as a sublimation residue
size_distribution	'powerlaw'	'single', 'powerlaw', 'file'
a_min, a_max	1, 1000	grain radii [μm]
q	3.5	exponent, d$N \propto a^{-q}$da
n_sizes	100	number of logarithmic size bins; for Mie polarisation images and for (sub)millimetre fluxes use at least 30 per decade (Sect. 21)
size_file		radius [μm] and relative number of grains per bin (increasing radii)
grain_shape	'sphere'	'sphere' (Mie) or 'dhs'
dhs_fmax, dhs_nf	0.8, 10	$f_{\max}$ ($0 < f_{\max} \leq 1$) and number of Gauss–Legendre points for 'dhs'
dhs_xmax	1000	'dhs': larger grains computed at this size parameter with the same kx (Sect. 4.3); 0 = full DHS
blowout	.true.	remove grains blown out by radiation pressure
blowout_release	'circular'	threshold $\beta_{\rm crit}$: 'circular' (1/2), 'pericentre' ($(1-e)/2$), 'apocentre' ($(1+e)/2$), with e of each belt
blowout_beta	(unset)	any threshold > 0; not together with 'pericentre' or 'apocentre'
sublimation	.false.	switch sublimation on
t_sublimation	-1	per component [K]; ≤ 0: never sublimates
sublimation_residue	0	per component: 0 = vanishes, n = becomes component n
photosputtering	.false.	switch ice loss by photosputtering on (Sect. 6.4)
ps_yield	0	per component: yield Y [molecules/photon]; 0 = no photosputtering
ps_ice_fraction	1	per component: ice mass fraction
ps_residue	0	per component: 0 = vanishes, n = becomes component n (without photosputtering)
ps_time	'shortest'	reference time: 'collision', 'pr', 'age', 'shortest'
ps_tau, ps_age	0	vertical optical depth; age [yr]
ps_absorption	'grain'	Q in eq. (24): 'grain' or 'unity'
ps_band	0.0912, 0.2431	UV band [μm]
ps_uv_file		UV spectrum (λ [μm], F_ν [Jy] at the star's distance)
ps_isrf	0	interstellar UV photons [cm^{-2} s^{-1}]
ps_orbit	'local'	'local' or 'release' (UV and P-R time of the orbit after release; ice per orbit in eccentric belts)
ps_orbit_bins	32	orbit bins per eccentric belt ('release')
tdep	.false.	per component: temperature-dependent data; optical_data is then the base name of the files <base>_<T>K[.ext] (Sect. 6.2)
tdep_mode	'nearest'	'nearest' set or 'linear' interpolation of n, k in T
tdep_dt	5	'linear': largest step of the interpolated data sets [K]

Name	Default	Meaning
`tdep_component`	0	input of version 2.0 (one component; not together with `tdep`)
`tdep_temperatures`, `tdep_files`		version 2.0: temperatures [K] and files of the data sets
`emt`	`.false.`	create a two-component mixture before the run
`emt_matrix`, `emt_inclusion`		files of the two materials
`emt_volume_fraction`		volume fraction of the inclusion, $0 < f < 1$
`emt_rule`	`'maxwell-garnett'`	`'maxwell-garnett'`, `'bruggeman'`, `'lichtenecker'`, `'looyenga'`
`emt_output`		output file in `data_dir`; use it in `optical_data`

17.7 &scattering

Name	Default	Meaning
`n_angles`	181	scattering angles between 0° and 180° (odd)
`phase_function`	`'mie'`	`'mie'`, `'hg'`, `'hg2'`
`g1`, `g2`, `w`	0, 0, 1	HG asymmetry parameters and weight of the first component
`p_max`	0	maximum degree of polarisation for HG

17.8 Optical data files

Every file has three columns: wavelength [μm], n, k. Lines that cannot be read as three numbers are skipped, and Fortran `D` exponents are accepted. Sub-directories are allowed (`optical_data = 'kim2019/porous_astrosilicate/as50vac50'`). The repository contains the astronomical silicate of Draine (2003), the water-ice and mixture data of Kim et al. (2019), water ice of Mastrapa et al. (2008, 2009) at 50 K (joined with the Kim et al. 2019 data outside the measured range) and, for pure ice, at 15–120 K (amorphous) and 20–140 K (crystalline), and amorphous pyroxene (Dorschner et al. 1995), amorphous carbon (Zubko et al. 1996) and organics (Henning & Stognienko 1996) taken from `optool`. The file `data/optical_constants/README.md` describes every file and the references to cite.

18 Output files

All files go to `output_dir`. `<ext>` is the suffix built from `output_name`.

18.1 Images

File	Content	Unit
`image_reem`	thermal emission	`image_unit`
`image_sca`	scattered stellar light	`image_unit`
`image_Q`, `image_U`	Stokes Q, U ($Q > 0$ along the image x axis)	`image_unit`
`image_pol`	polarised intensity $\sqrt{Q^2 + U^2}$	`image_unit`
`image_Qphi`, `image_Uphi`	azimuthal Stokes parameters	`image_unit`
`image_poldeg`	degree of polarisation PI/I (star excluded)	1

File	Content	Unit
`image_total`	thermal + scattered (+ star if `add_star`)	`image_unit`
`image_tau`	line-of-sight geometrical optical depth	1
`image_*_obs`	the same after PSF convolution and masking	`image_unit`
`profile_major_axis`	cut along the projected major axis through the model images (before PSF convolution)	`image_unit`

FITS files hold one cube with axes x, y and wavelength. The header gives the pixel scale (`CDELT1/2` in arcsec, `PIXAU` in au), the star pixel (`CRPIX1/2`), the wavelengths (`WAVE0001`, ...), `BUNIT`, the main model parameters and f_d (`FDUST`). **ASCII** images (`image_<quantity><ext>_#i.dat`) have N rows (y) and N columns (x); `np.loadtxt(file)[iy-1, ix-1]` gives one pixel. Pixel (i_x, i_y) is centred at $x_\mathrm{d} = (i_x - (N+1)/2)\Delta$, $y_\mathrm{d} = (i_y - (N+1)/2)\Delta$.

Comparing with observations. Set `position_angle` to the position angle of the disk measured from north through east, and display the images with the first pixel in the lower left corner (`origin="lower"` in matplotlib). North is then up and east to the left. Which side of the major axis is the near side follows from the sign of the inclination convention in Sect. 2; if the forward-scattering side of the model lies on the wrong side, add 180° to the position angle.

18.2 SED and tables

File	Content
`SED<ext>.dat`	wavelength [μm]; thermal, scattered, Q, U, polarised, dust, star, total [Jy] (also `.fits` with FITS output)
`optical_depth<ext>.dat`	f_d, cross section, τ_mean, largest radial optical depth in the mid-plane; then $\tau_\perp$ against distance: azimuthal mean, towards pericentre, towards apocentre
`Temperature_Distribution<ext>.dat`	distance [au], T for every component and size
`grain_size_distribution<ext>.dat`	component, a, bin edges, number of grains, $N\pi a^2$, mass, β
`phase_function<ext>.dat`	scattered and polarised flux of the whole disk in 1° bins of the scattering angle [Jy per bin], for every wavelength; written whenever scattering is computed. The sum over all bins equals the scattered flux of the SED. It combines the phase function of the grains with the amount of dust seen at each angle. In SED mode it is sampled by the 2° azimuth grid of the direct integration and is noisier
`cube_reem<ext>_#i.fits`, `cube_sca<ext>_#i.fits`	flux [Jy] of every volume element towards the observer (only with `save_3d_cubes`)
`cube_scattering_angle<ext>.fits`	scattering angle [deg] at the centre of every volume element
`summary<ext>.txt`	parameters, f_d, optical depths, SED, list of files, run time
`cache/<name>.miex`	saved optical properties (binary; written atomically)

19 Python interface

The examples of Sect. 16 show the interface in use: Sect. 16.1 for a first model and Sect. 16.4 for a more complete one. This chapter is the reference.

19.1 Running a model

```
import dms

m = dms.Model.from_file("examples/debris_ring.nml")   # or dms.Model(star={...}, disk
    ={...})
m.update(disk={"r_in": 60.0, "eccentricity": 0.1, "omega": 30.0},
         image={"n_pixels": 151, "psf": "gaussian", "psf_fwhm": 0.1})
res = m.run()                        # write_files=False, quiet=True by default

res.f_d, res.tau_mean                # fractional luminosity, mean optical depth
res.wavelength, res.sed["thermal"] # micron, Jy
img = res.image("Qphi", observed=True, unit="mJy/arcsec2")   # [n_wavelength, ny, nx]
m.write("model.nml")                 # input file for the command-line program
```

Parameter names, units and groups are those of the input file. Lists are Python lists, e.g. `disk={"n_belts": 2, "r_in": [5, 80]}`. An invalid input raises `dms.DMSError`, and the Python session keeps running. A power-law size grid with fewer than 25 bins per decade gives a `dms.SizeGridWarning` (Sect. 21).

19.2 The `Result` object

Attribute or method	Content
`wavelength`	observing wavelengths [μm]
`sed`	dict [Jy]: `thermal`, `scattered`, `Q`, `U`, `polarised`, `star`, `total`
`image(name, observed, unit)`	cube [n_λ, n_y, n_x]; names `thermal`, `scattered`, `Q`, `U`, `polarised`, `Qphi`, `Uphi`, `poldeg`, `total`
`images`	names of the available images
`tau_map`, `mass_map_kg`	line-of-sight optical depth, dust mass per pixel
`offsets("arcsec")`	pixel offsets along one axis
`f_d`, `tau_mean`, `tau_peak`	global dust quantities
`blowout_beta`	(smallest, largest) blow-out threshold used; (0, 0) if blow-out is off
`tau_radial`	largest radial optical depth in the mid-plane (> 0.1: optically thin assumption fails)
`tau_profile`	`r_au`, `mean`, `pericentre`, `apocentre`
`temperature`	`r_au`, `T[n_dist, n_component, n_size]`
`grains`	`a`, `edges`, `number`, `beta`
`species`	optical species (temperature-dependent data): `component`, `T`, `is_data`, `file`
`photosputtering`	`None`, or `r_ice_loss` [$n_{\rm size}$, $n_{\rm comp}$] (largest distance [au] with ice loss; -1: component without photosputtering), `photons_1au` [$\mathrm{cm^{-2}\,s^{-1}}$], and with orbit bins `a_ice_loss` (largest semi-major axis [au]) and `orbit_bins`
`luminosity_lsun`, `pixel_arcsec`, `pixel_au`	stellar luminosity, pixel size
`phase_curve(il, normalise)`	(θ [deg], $F_{\rm sca}$, $F_{\rm pol}$) of the whole disk in 1° bins [Jy per bin]
`cube(name)`	`"reem"`, `"sca"`: [n_λ, n_z, n_y, n_x] in Jy; `"angle"`: scattering angle [n_z, n_y, n_x] (only with `save_3d_cubes`)

19.3 Fitting and parallel runs

The Fortran state is global, so one model runs at a time in a Python process; OpenMP parallelises inside the run. For many models at once (grids, MCMC with `emcee`), use separate processes and set `OMP_NUM_THREADS=1` in each. The example below fits an observed SED on a small grid. The guard `if __name__ == "__main__"` is required on macOS and Windows, where `multiprocessing` starts new processes by importing the script.

```
import os
os.environ["OMP_NUM_THREADS"] = "1"          # before importing dms
from multiprocessing import Pool
import numpy as np
import dms

lam, f_obs, err = np.loadtxt("my_sed.txt", unpack=True)   # micron, Jy, Jy

def make_model(r_in=50.0, alpha=1.0):
    m = dms.Model.from_file("examples/debris_ring.nml")
    m.update(run={"mode": "sed", "load_optics": True, "save_optics": True,
                  "optics_cache_name": "fit"},
             wavelengths={"lambda": list(lam)},
             disk={"r_in": r_in, "alpha": alpha})
    return m

def chi2(params):
    sed = make_model(*params).run().sed["total"]
    return np.sum(((sed - f_obs) / err) ** 2)

if __name__ == "__main__":
    make_model().run()                       # computes and saves the optics once
    grid = [(r, a) for r in (40, 50, 60) for a in (0.0, 1.0, 2.0)]
    with Pool(8) as pool:
        values = pool.map(chi2, grid)
    print(grid[int(np.argmin(values))])
```

19.4 Other modules

- `dms.io`: read the output files of the command-line program (`read_image`, `read_sed`, `read_profile`, `read_optical_depth`).
- `dms.plot`: quick-look plots of a `Result` (`image`, `sed`, `temperature`).
- `dms.namelist`: `read_namelist`, `write_namelist`.
- `dms.stellar`, `dms.batch`, `dms.emc`: see the next section.

20 Grids of models, composite grains and stellar atmospheres

20.1 Batch runs: `dms.batch`

`dms.batch` runs many models from a queue folder, several in parallel, and survives interruptions.

```
import dms
from dms import batch

base = dms.Model.from_file("examples/debris_ring.nml")
```

```
jobs = batch.grid(base, {"disk.inclination": [0, 45, 70], "dust.q": [3.0, 3.5, 4.0]})
batch.write_queue(jobs, "queue")          # one input file per model
batch.run("queue", "runs", workers=8)     # returns when the queue is empty
table = batch.collect("runs")             # one row per model
```

From a terminal, which keeps running when a notebook is closed:

```
python -m dms.batch run queue runs -j 8 --watch   # also picks up files added later
python -m dms.batch status queue
python -m dms.batch requeue queue                 # failed models back into the queue
touch queue/STOP                                  # ends a --watch run
```

A file being computed moves from `queue/` to `queue/running/` and then to `queue/done/` or `queue/failed/`. Several runners, also on different machines sharing the folder, can work on one queue, because a file is claimed by an atomic rename. Each model writes into `runs/<name>/`: the DMS output, `input.nml`, `dms.log`, `meta.json` and `result.json` (wavelengths, SED, degree of polarisation against scattering angle, and the disk-integrated Q_ϕ/I if images were made). With `optics_cache_name = 'auto'`, models that differ only in geometry, star or q share one optical-property cache, and the runner never starts two models that would compute the same cache at the same time.

Parallel computing. There are two levels. Inside one model, DMS uses OpenMP threads (`omp_threads` of `batch.run`, i.e. `OMP_NUM_THREADS` of each run; a single run from Python with `Model.run` uses OpenMP alone). `dms.batch` runs `workers` models at the same time as separate processes. `workers` × `omp_threads` should not exceed the number of cores. Many small models run fastest with many workers and one or two threads each; a few large images, with fewer workers and more threads.

Section 16.12 gives a complete example.

20.2 Composite grains: `dms.emc`

`dms.emc.mix` runs `emc` (Ossenkopf 1991) without its interactive menus and writes a DMS optical-data file. The executable is given with `emc=...` or the environment variable `DMS_EMC`.

```
from dms import emc
C = emc.Component
emc.mix("bruggeman",
        [C("astrosilicate_draine03_862", 0.5, "sphere"),
         C("kim2019/pure_ices/composite_aice.txt", 0.5, "needle")],
        "data/optical_constants/mixtures/as_aice_bg.txt")
```

Theories are `'maxwell-garnett'`, `'bruggeman'`, `'uni-mg'` and `'uni-bg'`; shapes are `'host'`, `'sphere'`, `'needle'`, `'platelet'`, `'coated'` (with `mantle=`), `'coating'` and `'shape'` (with a depolarisation factor `L=`). The name `'vacuum'` selects the built-in vacuum of `emc`, for porous grains. `grid_from=i` takes the wavelength grid of the result from component i. For a grain made only of core and mantle, `'uni-mg'` and `'uni-bg'` give the same result, since both reduce to the coated-sphere formula.

20.3 Model-atmosphere grids: `dms.stellar`

A grid directory contains `grid_index.txt` (lines with $T_{\rm eff}$, $\log g$ and a file name) and one file per model with wavelength [μm] and surface flux F_λ [$\mathrm{W\,m^{-2}\,m^{-1}}$]. Grids are built with (internet access required):

```
python -m dms.stellar ck04    data/stellar_models/ck04_p00   --teff 4000 10000 --logg
     3.5 5.0
python -m dms.stellar phoenix data/stellar_models/phoenix_z0 --teff 3000 7000  --logg
     4.0 5.0
python -m dms.stellar blackbody data/stellar_models/bb --teff 3000 10000
```

and used with `&star spectrum = 'grid', grid_dir = 'data/stellar_models/ck04_p00', teff = 5770., logg = 4.44 /`. Please cite Castelli & Kurucz (2004) or Husser et al. (2013) for the grid you use. The download routines have not yet been tested against the archive servers; reading and interpolating a grid was tested with black-body grids.

21 Performance and accuracy

- **Optical properties dominate** a single run. Save them once (`save_optics`) and reuse them (`load_optics`) while the grains, wavelengths and optical data stay the same; the cache is ignored automatically when any of these change. DHS is 10–20 times slower than compact spheres.
- **SED first.** `mode = 'sed'` is much faster than images and gives the same SED.
- **Image resolution.** The time scales with the number of pixels and the refinement. Start with 101 pixels and $N_{\max} = 3$. Increase $N_{\max}$ for thin edge-on disks and narrow belts, and check that the result changes by less than your tolerance.
- **Temperature criterion.** `max_delta_t = 2` is adequate for most cases. For hot inner dust a smaller value refines more near the star.
- **Size bins.** Each size bin is represented by one grain size. Mie polarisation of large grains oscillates with grain size, and a coarse size grid leaves radial stripes in Q_ϕ images. The thermal emission at (sub)millimetre wavelengths comes from grains with $2\pi a \sim \lambda$, whose absorption efficiency changes quickly with λ/a; on a coarse grid the SED then ripples with the period of the size grid. For the quick-start model of Sect. 19 (1–1000 μm, $q = 3.5$, astronomical silicate) the largest deviation between 100 and 2000 μm from a 400-bin calculation is 29 % with 30 bins (10 per decade; 16 % at 870 μm), 12 % with 60 bins and 4 % with 100 bins. Use at least 30 bins per decade for Mie polarisation images and for (sub)millimetre fluxes; the default of 100 bins gives 33 per decade for 1–1000 μm. Below 25 bins per decade DMS prints a note, and the Python interface issues a `dms.SizeGridWarning`.
- **Angles.** Use more than 181 angles when large grains at short wavelengths make the forward peak narrow and the forward-scattering side of the disk matters.
- **Threads.** `OMP_NUM_THREADS` sets the number of threads of one run; for batches, prefer one thread per model and many models in parallel.

22 Troubleshooting

Message or problem	Cause and remedy
`input file not found`	Wrong path; paths are relative to the working directory.
`error in namelist group &disk`	Misspelt parameter name or a missing quote; the compiler message after the colon names the problem.

Message or problem	Cause and remedy
`observing wavelengths must lie within the wavelength range of the optical data`	Choose wavelengths inside the range shared by all optical-data files (printed in the message).
`no grains left: all grain sizes are blown out`	Every size bin has $\beta \geq 0.5$. Increase `a_min` or `a_max`, or switch `blowout` off.
`no grains left after sublimation / blow-out`	Check `t_sublimation`, the residues, `a_min` and the belt radii.
`no dust inside the model space`	The belt radii, opening angle or field do not contain dust; check the `&disk` parameters.
`blow-out requires a positive stellar mass`	Set `&star mass`.
`give either blowout_beta or blowout_release`	Use one of the two inputs.
`Mie calculation failed`	Size parameter beyond the limit of the Mie routine (very large grains at very short wavelengths, or very small grains at very long ones). Reduce the range.
`T_eff ... is outside the model grid`	Extend the grid or change $T_{\rm eff}$, $\log g$.
Images look blocky near the star	Increase `n_split_max` or decrease `max_delta_t`.
`ImportError: compiled DMS extension is missing`	Run `make python`; with Python 3.12 or later install `meson` and `ninja` first.
`make python` fails with OpenMP symbols	Build everything without OpenMP: `make serial`, then `make python OMP=""`. Otherwise make sure that `gfortran` and the compiler used by Python belong to the same toolchain.
Results differ between machines	Different compilers or `-march=native`; differences should be at the level of rounding.

23 Extending dms

23.1 Your own density distribution

Edit `src/user_density.f90` and recompile (`make`; for Python also `make python`). The function receives the semi-major axis $\mathcal{A}$ [m], the disk-frame coordinates X, Y, Z [m], the belt number and the ten parameters `user_par`, and returns a non-negative relative density. DMS applies the belt limits, the factor $\mathcal{A}/s$ of eccentric belts and the vertical profile itself. The default is a power law times a Gaussian clump:

```
psi   = atan2(X, Y)                                   ! azimuth in the disk plane
dpsi  = modulo(psi - par(3) * deg2rad + pi, 2.0_r2 * pi) - pi
width = max(par(4), 1.0e-3_r2) * deg2rad
rho   = (a / au)**(-par(1)) * (1.0_r2 + par(2) * exp(-0.5_r2 * (dpsi / width)**2))
```

If the distribution is symmetric under $Y \to -Y$, set `user_density_symmetric = .true.` to halve the computing time of images.

23.2 Adding an input parameter

1. Declare it in `parameters.f90`; set its default in `set_defaults` and check it in `finalize_input` (`input_setup.f90`).
2. Add it to the namelist in `read_input_file.f90` and, if relevant, to the interactive questions.

3. Release new allocatable arrays in `dms_reset.f90`.
4. Document it in `docs/input_reference.md` and in this manual.

Errors are reported with `call dms_fail("message")` followed by `return`; `stop` must not be used, so that the Python interface survives invalid input. The routines called inside the OpenMP loops must only read global variables. Every new feature gets a test in `tests/`. Before a pull request, `make debug` must compile without warnings and `make test` must pass; GitHub Actions repeats both.

24 Citing, licence and acknowledgements

If you use DMS, please cite Kim et al. (2018) and, if one is given for the release you use, the DOI of that release (see the repository page). If you use the optical data in `data/optical_constants/`, please also cite the references given there (for the water-ice data: Kim et al. 2019 and the original laboratory measurements). The file `CITATION.cff` lets GitHub show a citation for the repository.

Licence. DMS is distributed under the BSD 3-Clause licence (file `LICENSE`), with two exceptions. The Mie routine of `miex` (subroutines `aa2` and `shexqnn2` in `src/mie_routines.f90`) was written by Wolf & Voshchinnikov (2004) and published in the CPC Program Library (ADUD_v1_0) under the CPC Non-Profit Use Licence, which allows academic and non-profit use only; it is included in DMS with the written permission of S. Wolf, and was adapted for DMS 2.0 without changing the algorithm (Sect. 5). The optical data in `data/optical_constants/` come from the publications listed there; the files from optool are distributed under its MIT licence. The coated-sphere and DHS routines in `src/mie_routines.f90` were written for DMS and implement the algorithms of Yang (2003) and Min et al. (2005). The code `emc` (Ossenkopf 1991) is not distributed with DMS; `dms.emc` calls a copy installed by the user.

Acknowledgements. I thank Steve Ertel, Robert Brunngräber, Florian Kirchschlager, Sebastian Wolf and Thomas Stuber for their help with the original version of the code.

About the cover

All eight images on the cover were computed with DMS 2.0 at 401×401 pixels; they are in `docs/manual/cover/images/`, and the models are listed in the table below. All models use astronomical silicate (Draine 2003) with grain radii from 1 to 1000 μm, $q = 3.5$ and Mie theory. Only the display was added after DMS: every image is divided by a value close to its maximum (the 99.9th percentile of its pixels) and shown with a logarithmic or power-law stretch.

Panel	Model
Scattered light and Q_ϕ, 1.6 μm	Gaussian ring, $\mu = 78$ au, $\sigma = 3$ au, Gaussian vertical profile with $h/r = 0.02$, $i = 76°$; star with $T_{\rm eff} = 9500$ K at 72 pc. The near side (upper left) is brighter because the grains scatter mainly forwards.
Edge-on disk, 1.6 μm	Broken power law (`'broken'`), rising as $\mathcal{A}^5$ inside 90 au and falling as $\mathcal{A}^{-3}$ outside, $h/r = 0.05$, $i = 87°$.
Two belts, 24/70/870 μm	Inner belt 8–20 au (3 % of the dust mass) and outer Gaussian belt with $\mu = 140$ au, $\sigma = 10$ au and $e = 0.12$, $i = 55°$. Red, green and blue show 870, 70 and 24 μm; one logarithmic stretch of the summed intensity keeps the colour (Lupton et al. 2004).
Pericentre and apocentre glow, 24 and 870 μm	Ring with $\mathcal{A} = 136$–140 au, $e = 0.3$, $i = 20°$, at 7.7 pc; both images convolved by DMS with a Gaussian PSF of 1.5″ FWHM (`psf = 'gaussian'`), wider than the ring. The apocentre-to-pericentre ratio of the peak brightness is 0.22 at 24 μm and 1.24 at 870 μm (Sect. 2). Apocentre glow was first detected conclusively in the ALMA map of Fomalhaut (MacGregor et al. 2017).
Belt with a gap, 870 μm	User-defined density (`density_model = 'user'`, Sect. 23): $n \propto 1 - 0.5 \exp[-(\mathcal{A} - 80\,\mathrm{au})^2/2(10\,\mathrm{au})^2]$ between 40 and 140 au, similar in size to the gap in the belt of HD 107146 (Marino et al. 2018).
Ring with a clump, 870 μm	The example shipped in `src/user_density.f90` (Sect. 23): $n \propto \mathcal{A}^{-1}[1 + 2\exp(-\Delta\psi^2/2 \cdot 25°^2)]$, ring 50–90 au. Planets that migrate outwards can trap planetesimals in mean-motion resonances and produce such clumps (Wyatt 2003).

References

Andersson S., Al-Halabi A., Kroes G.-J., van Dishoeck E. F., 2006, J. Chem. Phys., 124, 064715

Augereau J.-C., Lagrange A.-M., Mouillet D., Papaloizou J. C. B., Grorod P. A., 1999, A&A, 348, 557

Bohren C. F., Huffman D. R., 1983, Absorption and Scattering of Light by Small Particles. Wiley, New York

Burns J. A., Lamy P. L., Soter S., 1979, Icarus, 40, 1

Castelli F., Kurucz R. L., 2004, arXiv:astro-ph/0405087

Dorschner J., Begemann B., Henning Th., Jäger C., Mutschke H., 1995, A&A, 300, 503

Draine B. T., 2003, ApJ, 598, 1026

Dullemond C. P., Juhasz A., Pohl A., et al., 2012, RADMC-3D, Astrophysics Source Code Library, ascl:1202.015

Engler N., et al., 2017, A&A, 607, A90

Ertel S., Wolf S., Metchev S., et al., 2011, A&A, 533, A132

Ertel S., Wolf S., Marshall J. P., et al., 2012, A&A, 541, A148

Grigorieva A., Thébault Ph., Artymowicz P., Brandeker A., 2007, A&A, 475, 755

Henning Th., Stognienko R., 1996, A&A, 311, 291

Henyey L. G., Greenstein J. L., 1941, ApJ, 93, 70

Husser T.-O., et al., 2013, A&A, 553, A6

Kim M., Wolf S., Löhne T., Kirchschlager F., Krivov A. V., 2018, A&A, 618, A38

Kim M., Wolf S., Potapov A., Mutschke H., Jäger C., 2019, A&A, 629, A141

Lupton R., Blanton M. R., Fekete G., Hogg D. W., O'Mullane W., Szalay A., Wherry N., 2004, PASP, 116, 133

MacGregor M. A., Matrà L., Kalas P., et al., 2017, ApJ, 842, 8

Marino S., Carpenter J., Wyatt M. C., et al., 2018, MNRAS, 479, 5423

Mastrapa R. M., Bernstein M. P., Sandford S. A., Roush T. L., Cruikshank D. P., Dalle Ore C. M., 2008, Icarus, 197, 307

Mastrapa R. M., Sandford S. A., Roush T. L., Cruikshank D. P., Dalle Ore C. M., 2009, ApJ, 701, 1347
Min M., Hovenier J. W., de Koter A., 2005, A&A, 432, 909
Min M., Rab C., Woitke P., Dominik C., Ménard F., 2016, A&A, 585, A13
Ossenkopf V., 1991, A&A, 251, 210
Pan M., Nesvold E. R., Kuchner M. J., 2016, ApJ, 832, 81
Peña O., Pal U., 2009, Comput. Phys. Commun., 180, 2348
Pinte C., Ménard F., Duchêne G., Bastien P., 2006, A&A, 459, 797
Robitaille T. P., 2011, A&A, 536, A79
Westley M. S., Baragiola R. A., Johnson R. E., Baratta G. A., 1995, Planet. Space Sci., 43, 1311
Wolf S., Voshchinnikov N. V., 2004, Comput. Phys. Commun., 162, 113
Wyatt M. C., 2003, ApJ, 598, 1321
Wyatt S. P., Whipple F. L., 1950, ApJ, 111, 134
Yang W., 2003, Appl. Opt., 42, 1710
Zubko V. G., Mennella V., Colangeli L., Bussoletti E., 1996, MNRAS, 282, 1321